智调盒——AI 上板摆渡站

摘要

大模型已经能够辅助生成和修改嵌入式代码，但从“给出代码建议”到“在真实硬件上验证修改是否有效”，仍存在一道明显的工程断层。**“智调盒”并非单纯的自动下载器，而是尝试让 AI 真正进入真实硬件调试闭环：从自然语言目标出发，完成目标工程构建、目标板烧录、运行信息采集与物理行为测量，并根据真实证据继续分析和迭代。**

系统由 PC、Zynq-7000 与 STM32G431RBT6 协同组成。PC 运行 AI Agent、确定性 Controller 及可视化控制台；Zynq PS 基于定制 Linux 环境承担部署、OpenOCD 和板端访问；FPGA PL 则利用可编程 I/O 与可扩展硬件接口，作为独立于目标固件的物理观测层，以 100 MHz 时钟测量 GPIO 完整周期和高电平时间，并通过 AXI UART Lite 采集 UART 运行信息。四个稳定接口分别完成目标 MCU 工程构建、固件烧录、日志读取与硬件测试。需要特别说明的是，**闭环中反复构建和烧录的是 STM32 等目标 MCU 的固件；Zynq/FPGA 在完成部署与 PL 初始化后，不随每一轮调试任务重复烧录。**

每轮执行均保留源码与固件 SHA、工具审计、UART 健康状态及 PL 新边沿证据。AI 负责理解目标、分析证据和提出源码修改，Controller 则约束执行范围、工具顺序、预算、人工审批与停止条件；工具执行成功与目标物理达标分别记录，最终 PASS/FAIL 由确定性硬件规则给出，而非由模型自行宣布。

在“将 STM32 PA1 完整周期调节为 1000 ± 50 ms”的真实闭环实验中，首轮 PL 实测周期为 601.7169 ms，判定 FAIL。模型结合源码与物理测量结果提出将 HAL_Delay(300) 修改为 HAL_Delay(500)；人工批准后重新构建并烧录目标 MCU，第二轮 PL 独立测得周期 1001.08682 ms，绝对误差 1.08682 ms，最终判定 PASS。项目同时提供五类可复用 Skill、工程代码、验证证据、HDF 及 Linux 定制与部署说明，以支持结果追溯与工程复现。

目录

第一部分　作品概述 4
1.1 功能与特性 4
1.2 应用领域 4
1.3 主要技术特点 4
1.4 主要性能指标 5
1.5 主要创新点 5
1.6 设计流程 5
1.7 与赛道评测要点的对应关系 5
第二部分　系统组成及功能说明 8
2.1 整体介绍 8
2.1.1 设计背景与需求分析 8
2.1.2 总体系统架构 8
2.1.3 系统数据流与闭环流程 9
2.1.4 软硬件划分依据 10
2.2 硬件系统介绍 11
2.2.1 硬件整体组成 11
2.2.2 机械与外观设计 11
2.2.3 硬件连接方式与测试接口 12
2.2.4 PL GPIO 周期测量模块 13
2.2.5 PL UART 日志观测模块 15
2.2.6 LED 状态指示模块 16
2.2.7 AXI 地址与寄存器映射 16
2.2.8 当前 HDF 硬件交接文件 16
2.2.9 并列 STM32 LED 同步演示工程 17
2.3 软件系统介绍 18
2.3.1 软件整体架构 18
2.3.2 AI Agent / Orchestrator 19
2.3.3 Reviewed Controller 19
2.3.4 四个稳定工具接口 20
2.3.5 RealBackend 与 Observation 21
2.3.6 人工审批与 Diff 安全机制 21
2.3.7 PC 端可视化控制台 22
2.3.8 LED workflow 状态同步 22
2.3.9 Linux 基础系统与 rootfs 定制部署 22
第三部分　完成情况及性能参数 26
3.1 整体介绍 26
3.2 工程成果 28
3.2.1 硬件与 PL 成果 28
3.2.2 电路 / 接口成果 28
3.2.3 软件成果 29
3.2.4 交付目录、命名与输入身份 30
3.3 特性成果 31

3.3.1 四个公开接口真机验证 31
3.3.2 完整 AI 调试闭环实验 32
3.3.3 功能正确性与异常验证 37
3.3.4 性能与资源指标 38
3.3.5 优化过程及前后对比 40
3.3.6 大模型协作记录 40
3.3.7 Skill 提炼过程与验证 43
3.3.8 从零复现说明 46
3.3.9 HDF 与 Linux 定制的离线验证 49
第四部分 总结 50
4.1 可扩展之处 50
4.2 心得体会 50
第五部分 参考文献 52
附录 A 素材与证据路径清单 54
附录 B 文档、代码与验证证据对照 57

第一部分　作品概述

版本基准：本文工程实现、验证证据与复现路径对齐 https://github.com/Aurora-wwj/zhitiao_box 的代码与证据基准提交 9f2e1bb71a072e667a3ae55777ca097deface748，核对日期为 2026 年 11 月 4 日。代码路径均相对该提交的仓库根目录；历史 JSON 中的原路径和截图原貌保留，当前发布路径通过映射对应。五大部分沿用官方模板，新增技术小节及附录用于对照代码和证据。

1.1 功能与特性

用户输入可量化调试目标，确认工程、可修改文件及任务范围后，系统依次构建固件、上传并校验、烧录复位、采集 UART 和测量物理行为，其中反复烧录的是目标 MCU 固件，Zynq/FPGA 作为调试与观测基础设施，在完成部署初始化后不随每轮任务重复配置。目标未满足时，AI 根据本轮证据提出源码 diff；人工批准后应用修改并进入下一轮，直到达标或触发停止条件。PC 控制台展示目标、阶段、UART、AI 摘要和 Before / Target / After 结果，支持报告下载与受控恢复。可选四灯显示四工具进度；LED 表示执行状态，不代替被测目标判定。

1.2 应用领域

作品面向 MCU 参数调试、板级验证、MCU / FPGA 联调和实验室自动化，也可用于嵌入式教学中展示“源码修改—真实执行—证据反馈”的全过程。研发原型、FAE 支持及共享实验室属于应用方向。当前已完成 STM32G431 数字输出完整周期调节的验证；推广到其他目标、外设或远程服务，仍需相应接线、后端适配与独立验收。

1.3 主要技术特点

系统以 Zynq PS / PL 协同实现网络桥接和独立观测：PL 提供 100 MHz 周期 / 高电平计数、新边沿和有效位，AXI Uartlite 接入 UART；PC 端采用 Agent 与确定性 Controller 分工，执行顺序、权限和预算不由模型决定。这种可编程、并行且可扩展的硬件接口能力，使 FPGA 不只是通信桥，而能够直接成为真实物理行为的测量层。四工具统一输出结构化结果，保留源码 / 产物 SHA、观测来源和审计轨迹。会话范围审批与每个新 diff 审批相结合，并由 Web UI 及可选 LED 状态适配器呈现过程。

1.4 主要性能指标

表 1-1 主要性能指标

指标	已确认结果
目标完整周期	1000 ± 50 ms
第一轮	601.7169 ms，FAIL
第二轮	1001.08682 ms，PASS
最终绝对误差	1.08682 ms
PL 观测时钟	100 MHz
UART	115200，8N1
闭环与新边沿	2 轮；每轮新增 2 边沿

1.5 主要创新点

（1）将大语言模型（LLM）正式纳入真实硬件调试闭环，把 AI 源码建议连接到真实构建、烧录、物理验证及失败再迭代；

（2）以确定性 Controller 约束模型行动，人工审阅新 diff，物理规则判 PASS；

（3）结合 UART 辅助证据、PL 独立测量和 freshness 检查形成可追溯反馈；

（4）以四个稳定接口和主题 Skill 沉淀跨任务复用的工程流程与边界；

1.6 设计流程

确认目标与范围后按构建、烧录、UART、PL 顺序执行。仅在目标未达标时分析证据并提出源码修改；每个新 diff 均需人工批准。工具故障、拒绝、取消、次数或预算耗尽会停止流程。

1.7 与赛道评测要点的对应关系

根据 AMD 自主选题赛道评测指标，本文将作品的主要设计与实测证据按照五类评测要点建立对应关系。下表仅用于帮助读者快速定位相关内容，不作自行评分；具体结论仍以后文工程实现、原始记录与实测数据为依据。

评测要点	权重	本作品主要对应内容与证据
创新性与应用价值	30	**将大模型建议连接到真实构建、烧录、物理观测与失败再迭代；以 FPGA PL 作为独立于目标固件自报信息的物理观测层，并以确定性 Controller 与人工审批约束 AI 行动。面向 MCU 参数调试、板级验证、软硬件联调和实验室自动化等场景。**
性能与资源优化效果	20	以 100 MHz PL 时钟完成周期与高电平计数；报告供应商基线与最终观测工程的 LUT、FF、BRAM、DSP、IOB 及 routed timing 数据，用于量化新增观测能力的资源代价与时序余量；同时保留工具耗时和真实闭环测量结果。该比较用于说明性能与资源权衡，不将其表述为严格的同功能资源压缩实验。
功能正确性与设计完整性	20	**完成真实 build→flash→UART→PL 两轮闭环：**第一轮 601.7169 ms 判 FAIL，经 AI 提议与人工审批修改后，第二轮 1001.08682 ms 判 PASS；保留源码、固件 SHA、UART 健康状态、freshness、审批和审计记录，并通过异常回归与实物状态指示补充验证。
大模型协作与技能包	15	**将大语言模型（Large Language Model, LLM）实际接入硬件调试闭环**，用于理解自然语言目标、分析构建 / 烧录 / UART / PL 返回的真实证据并提出下一轮源码修改建议；LLM 不拥有工具执行、自审批或最终 PASS / FAIL 判定权限。项目同时保留提示模板、模型理由、实际源码提案、人工审批及重新验证记录，并将开发过程中的失败与观察提炼为五项可复用 Skill。
文档质量与可复现性	15	以固定提交、文件 SHA、HDF、构建与部署脚本、Linux 定制说明、从零复现步骤、原始 JSON 和文档—代码—证据对照表建立追溯链；工程目录与验证入口在公开仓库中统一组织。

上述五类内容并非彼此独立：**例如真实闭环结果同时体现功能正确性、创新性和大模型协作价值**，但本文在后续章节中按照各自技术问题分别提供证据，避免用同一结果替代不同维度的验证。

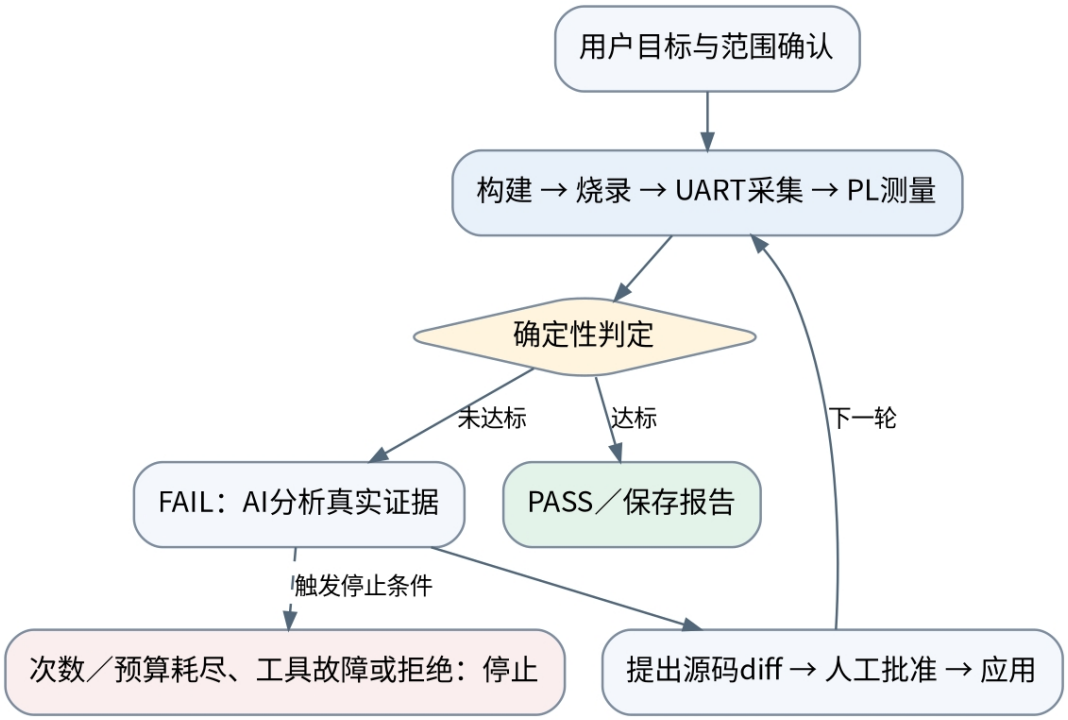

图 1-1 受控硬件调试闭环；v5 预加载属于任务前初始化。

第二部分　系统组成及功能说明

2.1 整体介绍

2.1.1 设计背景与需求分析

传统 MCU 调试已具备编译器、调试探针、OpenOCD 及串口工具，但通常由开发者手工连接多个操作，比较修改前后行为。一般脚本可以减少重复命令，固定顺序本身却不包含“分析失败证据后修改源码”的决策环节；单独的 AI 编程助手能够给出修改建议，也需要外部机制确认这些建议在实物上是否成立。本作品的改进重点是将这两种能力组织成可审计、可停止、有人控制的执行闭环，而不是替代已有编译器或调试器。因此，**本作品的最终目标并不是把既有烧录流程自动化，而是让 AI 的修改能够真正进入目标硬件，并获得独立于目标程序自报信息的物理反馈。**[2-4,8-11]

需求分为三类：第一，目标必须转化为明确被测信号、期望值与容差，避免把翻转间隔误当完整周期；第二，模型只提出建议，工程路径、执行次数、超时和审批必须由可信程序控制；第三，使用独立于目标固件自报文本的 PL 计数来验证物理行为，并记录本次观测是否确有新边沿。当前验证实例为 PA1 完整周期 1000±50 ms，UART 用作运行状态的辅助证据，不承诺任意外设自动初始化。

另外，**波形可以很好地成为 AI 与真实硬件物理行为之间的一种通用证据载体。**从时域观测角度，许多硬件输出，无论数字还是模拟，都可以表现为一个或多个随时间变化的电压或电流波形。

2.1.2 总体系统架构

系统采用“用户任务 — Agent / Orchestrator — Controller — 四工具 — RealBackend / Observation — 真实设备”的层次结构。PC 与 Zynq 通过以太网及 SSH / SCP 通信；Zynq 通过 USB ST-Link / SWD 连接 STM32，并在 PL 侧接收被测数字输出和 UART TX。观测结果以结构化字段回到确定性判定与 AI 上下文，所有关键阶段进入报告和事件流。

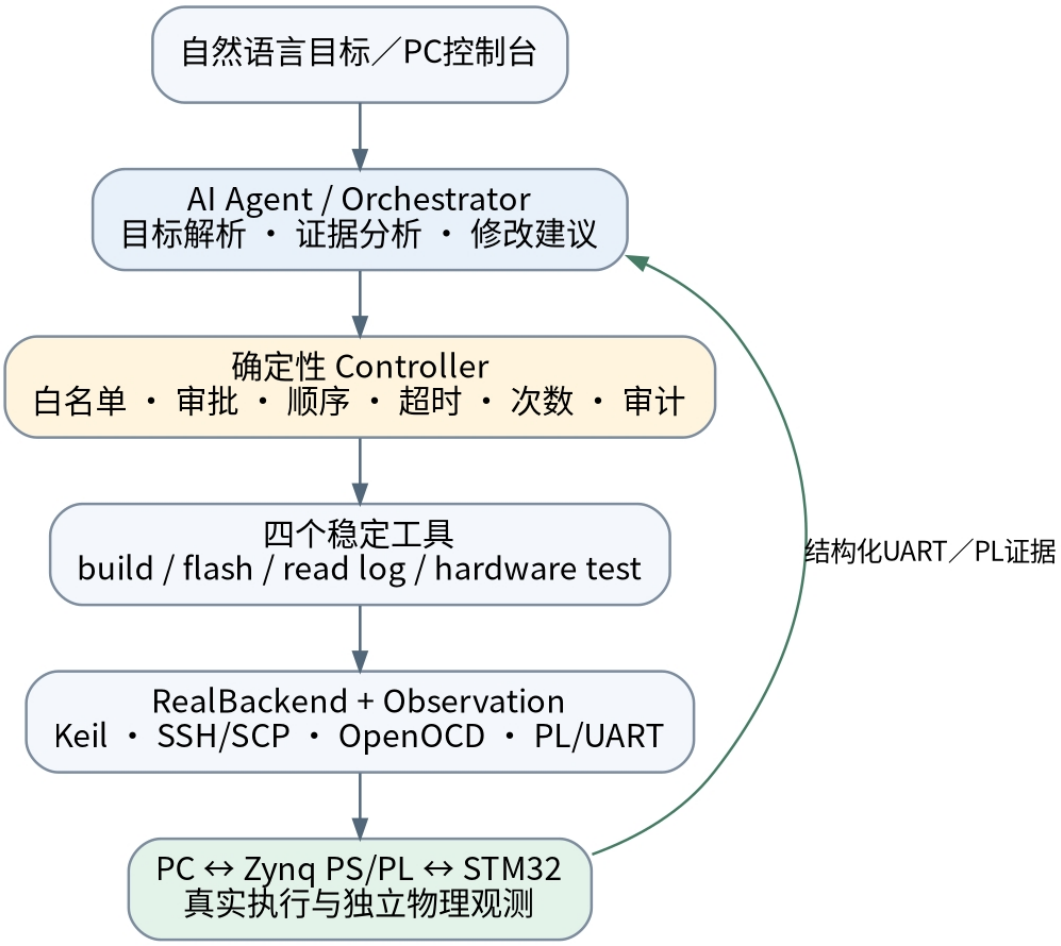

图 2-1 系统逻辑架构；箭头表示调用和证据反馈，不表示模型直接获得底层命令权限。

2.1.3 系统数据流与闭环流程

任务范围绑定允许访问的工程和源文件；构建前后核对输入身份，生成新 HEX 并计算 SHA-256。烧录阶段上传该产物并对比远端 SHA，再进行 program / verify / reset，防止混用旧文件。UART 采集和 PL 测试分别输出软件日志、健康信息与物理计数。Controller 区分工具成功和目标判定；若物理结果 FAIL，Orchestrator 将本轮证据及允许修改的源码交给模型，形成带 before_sha256 的完整文件替换提案，并在人工批准后重新绑定下一轮输入。

每轮 attempt 都是一次新的真实验证，原有 Controller 不负责“凭重复调用自动变聪明”。模型决策和源码编辑位于外层；内部仍采用确定性工具计划。审计记录中的输入、输出、源码 SHA、固件 SHA、观测来源、审批身份和停止原因共同构成数据链。

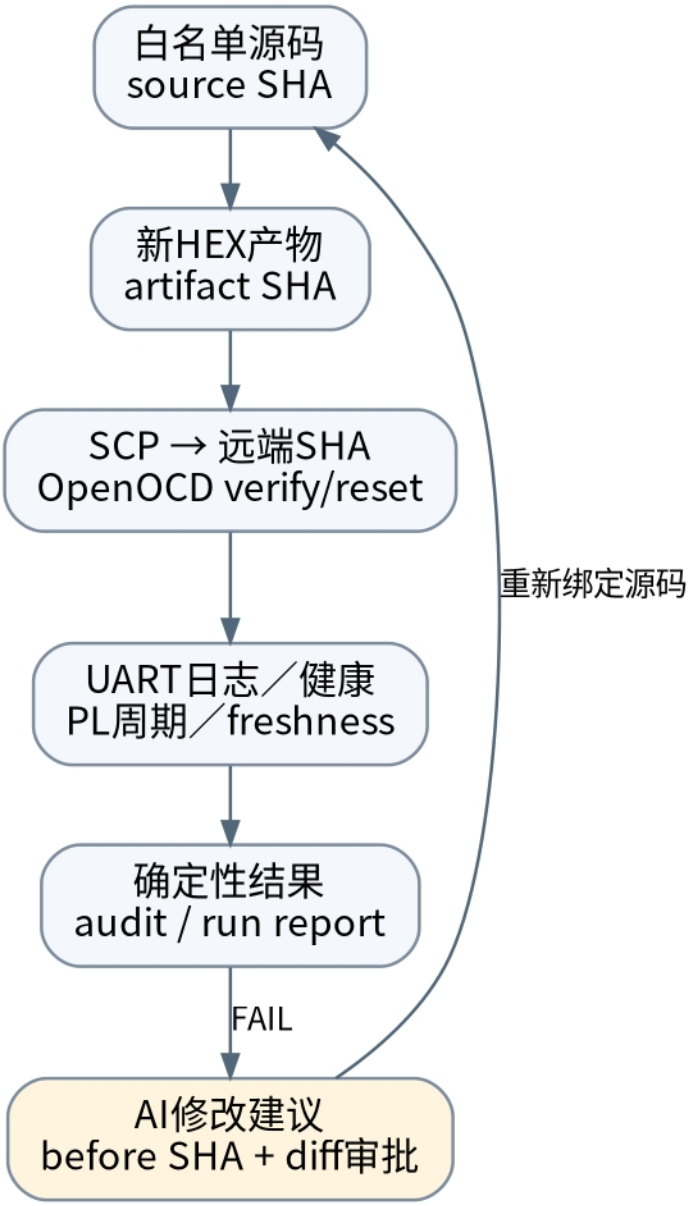

图 2-2 源码、产物、物理证据与审阅 diff 之间的数据关系。

2.1.4 软硬件划分依据

表 2-1 软硬件划分依据

单元	实际职责	划分依据
PC Agent / UI	目标理解、模型请求、修改建议和人机交互	软件上下文与模型接口复杂，适合上位机
PC Controller	路径、审批、顺序、预算、停止和审计	执行规则需要确定性，不交给 LLM 自行解释
Zynq PS / Linux	SSH / SCP、OpenOCD、工具和寄存器访问桥接	靠近目标板，复用成熟 Linux 工具链

单元	实际职责	划分依据
Zynq PL	GPIO 周期 / 高电平计数、UART 接入、LED 输出	可编程 I/O 直接接触真实物理信号，适合承担独立、并行且确定性的硬件测量
STM32G431	执行被调试固件并输出可观测行为	作为实际 DUT，软件行为由外部测量确认

NiceCode 模型通过 PC 端调用，请求模型标识为 gpt-5.6-sol、协议为 openai-responses。这里记录的是服务请求配置，未验证远端模型权重身份。当前未在 FPGA 上运行 LLM，也未使用 PYNQ Overlay，不以这些未实现能力作为结果。[1,14]

2.2 硬件系统介绍

2.2.1 硬件整体组成

表 2-2 硬件整体组成

部件	已采用的型号 / 设置	作用
Zynq 平台	ALINX AX7Z020；xc7z020clg400-2	PS 网络与部署，PL 观测和状态输出
目标 MCU	以 STM32G431RBT6 为例	执行 GPIO_PL_Test 固件
调试探针	USB ST-Link/V2，SWD	OpenOCD 烧录、校验和复位
上位机	Windows PC；Python 3.13、Keil MDK	Agent、编译、审批与浏览器 UI
通信	Ethernet；SSH 22；SCP	PC 与板端任务及文件传输
FPGA 工具	实际 Vivado 2017.4	原工程综合、实现及 v5 位流

AMD 指南推荐 2026.1 或 2025.2，但允许其他版本并要求说明与可复现性。本作品沿用板卡参考工程的 Vivado 2017.4 环境，并提供工程源输入、当前 HDF、外部 IP 清单及 Tcl 构建入口。本文资源与时序结果采用已归档的实际 Vivado 综合与实现报告，离线脚本检查不替代真实硬件构建结果。[1,3-5,13]

2.2.2 机械与外观设计

当前材料对应由开发板、ST-Link 和连接线组成的功能验证原型，未提供独立外壳 CAD、自研 PCB 或机械加工资料，因此本版报告不主张完成机械产品设计。模块布局应保证供电、共地、散热和探针连接可靠。外壳固定、接口标识与

抗误插设计属于后续产品化方向。

2.2.3 硬件连接方式与测试接口

通用接法是将被测 MCU 的某个数字输出连接到当前位流的 M18 观测入口，将某个 UART TX 连接到 M17，并共地。PA1、PA2 是本文 STM32 实例使用的引脚，不是产品唯一允许的 MCU 端物理接线。PL 只测输入端实际电平；更换被测 GPIO 仍需用户接线、目标固件配置及与真实信号一致的部署标签。M18 / M17 由当前 v5 XDC 固定，修改 Zynq 侧管脚需要重新生成并验证位流，不属于运行时自动路由。[6-9]

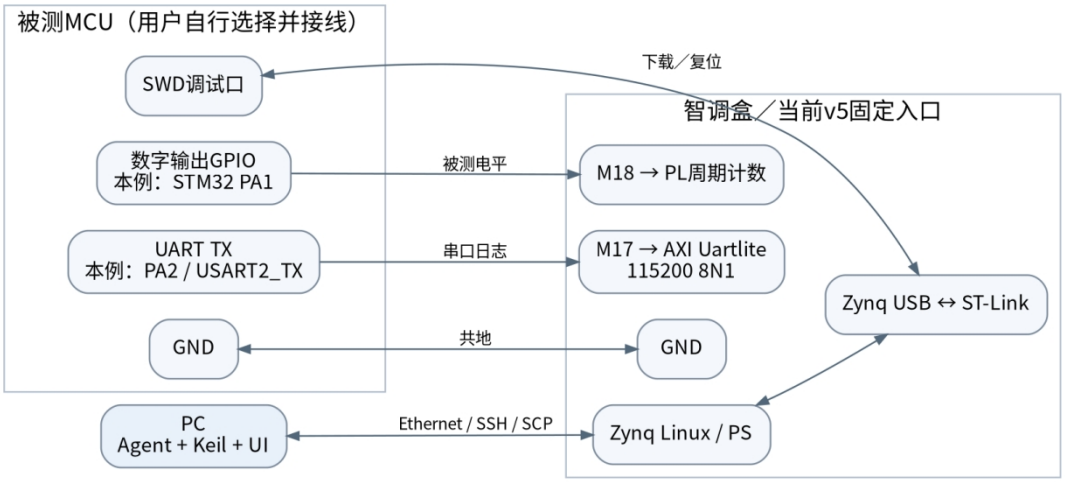

图 2-3 通用 MCU 接入及本例连接关系示意。

表 2-3 硬件连接方式与测试接口

功能	本文被测端	智调盒端	说明
周期观测	STM32 PA1	M18 / PL GPIO IN	本文完整闭环的数字信号
UART 日志	PA2 / USART2_TX	M17 / PL UART RX	115200，8N1；不自动识别任意波特率
烧录 / 复位	STM32 SWD	ST-Link → Zynq USB	OpenOCD program / verify / reset
控制通信	PC Ethernet	Zynq Ethernet	SSH / SCP；本例 192.168.10.1/24 与.2/24
参考地	MCU GND	Zynq GND	接线前确认电平兼容和共地

observed_gpio 和 stm32_tx_gpio 是受支持部署中的语义标签，不会把物理信号自动切换到指定 MCU 引脚。当前实例与电平标准已经按原 XDC、配置及实测报告核对；其他板卡必须另行确认供电和电气条件。

2.2.4 PL GPIO 周期测量模块

2.2.4.1 100 MHz 周期 / 高电平计数

模块 zhidiao_measure_core 以 100 MHz 时钟采样同步后的输入，使用 32 位自由运行时间戳计算相邻上升沿之间的完整周期，并在下降沿记录高电平持续时间。按标称时钟换算：T_ms = period_ticks×1000 / 100000000；H_ms = high_ticks×1000 / 100000000。一个计数 tick 为 10 ns，这是标称计数分辨率，不是经校准的绝对测量精度。时钟误差、异步采样及同步延迟仍需在精度扩展时考虑。[2,6]

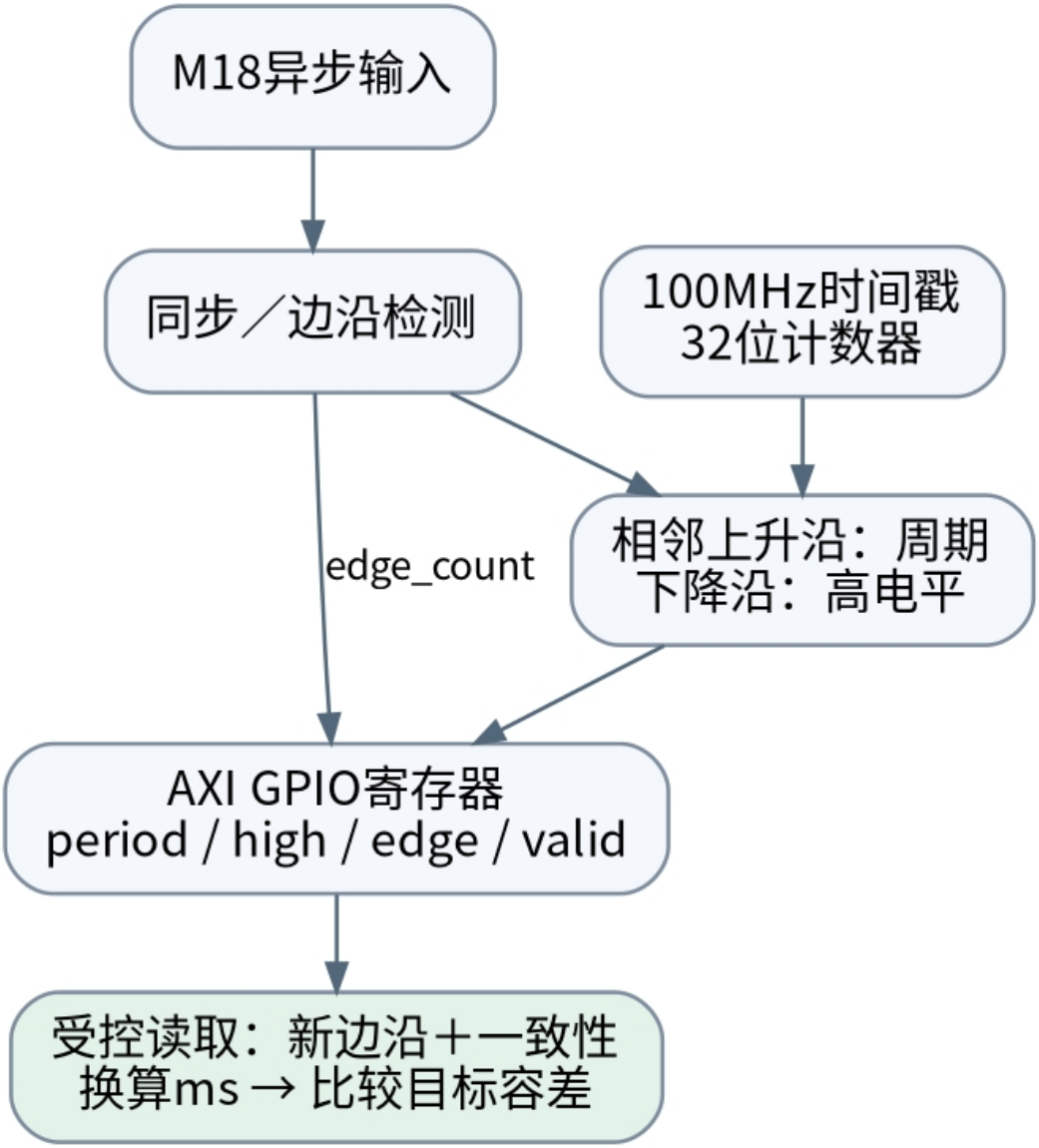

图 2-4 周期观测的数据通路示意。

2.2.4.2 edge_count 与 valid

上升沿和下降沿都会增加 edge_count；有效周期由相应边沿间时间差生成。period_ticks、high_ticks 和 valid 通过 AXI 寄存器供 PS 读取。valid 在产生一次有效结果后保持到复位，输入停止时旧测量也会保留。因此非零寄存器或 valid=1 不能单独证明当前固件、本次窗口产生了新的可信结果。

2.2.4.3 Freshness 机制

PLPeriodReader 先记录本轮边沿计数，再在有限窗口中等待至少两个新边沿，并用前后计数夹住周期 / 高电平 / valid 快照，检查一致性、有效性和数值范围。无新边沿或不一致时不判目标通过。最终两轮分别记录 fresh_edges=2、

freshness_verified=true；32 位 100 MHz 时间戳约 42.95 s 回绕，不能将超出可解释计数范围的间隔当作正确周期。

原 RTL 另有启动边界：同步器复位为 0，外部输入已为 HIGH 时，首次 valid 可能包含启动残段；仅两个新边沿并不保证所有启动相位都安全。本次保留黄金 RTL / 位流，不用修改原件掩盖边界。后续若要求严格拒绝首次残段，应单独修订 RTL、重生成位流并重新实测。[14]

2.2.5 PL UART 日志观测模块

2.2.5.1 AXI Uartlite

当前 PL 使用 AXI Uartlite，UART 配置为 115200、8 数据位、无校验、1 停止位，MCU TX 接入 M17。PLUARTReader 有独立 UARTObserverConfig，避免把周期信号标签当作 UART 来源。当前不承诺运行时任意波特率自动识别，换波特率或管脚时需要相应软硬件配置及验收。[7]

2.2.5.2 RX FIFO 读取

采集前在受审批计划内读取旧 sticky 状态，并对 Control 写 0x2 清 RX FIFO，排除历史字节；随后一次 SSH 内连续读取 FIFO，仅在空闲时按整数微秒 usleep 等待。默认采集窗口 5 s，字节上限 1024。清 FIFO 是实际写操作，必须处于 Controller 批准的范围内；它不是第五个公开工具。

2.2.5.3 Monotonic timestamp

采集窗口使用板端 / proc / uptime 提供的单调时间计算，日志时间戳描述本窗口内的读出时刻，不冒充 UART 线上字节到达的硬件时间戳。计时采用真实时间而非轮询次数累加，并受 PC 传输预算约束。真实两轮 capture_elapsed_ms 分别为 5000 和 5020，不能把业务窗口与含 SSH 往返的 duration_ms 混为一谈。

2.2.5.4 Health / error flags

结果保留 overflow、truncated、partial_tail、capture_end_reason、decoding_errors、frame_errors、parity_errors、bytes_captured 和 fifo_reset。正常 deadline 恰好截到未换行末行时标记 partial_tail，不作为致命证据丢失；max_bytes 触限、真正传输截断及健康错误仍停止。第一轮真实结果为 3 条完整 ZHIDIAO_OK 加 ZHIDIAO_O 半行，partial_tail=true、truncated=false、结束原因为 deadline，后续 PL 正常执行。

2.2.6 LED 状态指示模块

v5 中新增 4-bit AXI GPIO，Data 地址 0x43C60000，bit0～3 对应 LED1～4，active-low，默认 0xF 全灭；XDC 依次为 J14 / K14 / J18 / H18，LVCMOS33。四灯分别映射 build_firmware、flash_firmware、read_target_log、run_hardware_test。未开始灭、执行中普通闪烁、工具成功常亮、工具异常双闪；新 attempt 的 BUILDING 事件清空上轮进度。物理测试 success=true 但 passed=false 时，LED4 仍表示“测量工具完成”，不能当作 DUT 的 PASS 灯。

当前资料具备 RTL / BD / XDC、状态适配代码和离线验证，最终运行配置也启用了该适配器。状态写失败只形成 warning，不改变四工具或 DUT 判定。

2.2.7 AXI 地址与寄存器映射

表 2-4 AXI 地址与寄存器映射

地址	寄存器 / 作用	访问说明
0x43C30000	period_ticks	周期计数读取
0x43C30008	high_ticks	高电平计数读取
0x43C40000	edge_count	判断本轮新边沿
0x43C40008	valid	结合 freshness 读取
0x43C50000	UART RX FIFO	连续 drain 当前窗口字节
0x43C50008	UART Status	FIFO 及错误状态
0x43C5000C	UART Control	0x2 清 RX FIFO
0x43C60000	LED Data	固定 4-bit active-low 输出

表中仅列本文工作流使用的寄存器；底座摄像头 / HDMI 及未被本流程调用的其他地址不在此表，完整设计以 BD 和 HDF 的 system.hwh 为准。这些地址对应当前 v5 设计，不能用于未知 PL 镜像。prog_done=1 仅说明配置完成；磁盘文件 SHA 与 check_only 检查也不认证运行中的镜像身份。实际加载策略、设备节点和镜像来源必须由部署者确认。

2.2.8 当前 HDF 硬件交接文件

当前硬件交接文件为 src/fpga/zhidiao_measure/system_wrapper.hdf，由最终 v5 Vivado 工程导出，按 808369 字节原件保存，SHA-256 见附录 A。其 sysdef 标识 Vivado 2017.4、器件 xc7z020clg400-2，内部包含 HWH、BD Tcl、PS 初始

化文件及 bitstream 等内容，共 10 个成员。sim/linux_check.py 逐项检查成员 SHA 与 CRC，并核对周期、状态、UART、LED 的 AXI 地址、UART115200 及四灯宽度 / 默认 0xF。[14]

HDF 内 system_wrapper.bit 与 build/fpga/zhidiao_observe_v5_led.bit 逐字节相同，内 system_bd.tcl 与公开 BD 导出文件相同；PS 初始化 c / h / tcl 已与上传 vendor-base 交接文件比较一致。因此本 HDF 用于交接当前 v5 设计，不把另一个基础工程的 HDF 冒充最终导出。此前公开 Git 树没有跟踪 HDF，本次加入的是当前唯一交接版。HDF 标注的 Post-Synthesis 及内部资源字段不替代 placed / routed 原报告，资源表仍采用 3.3.4 所列原始实现报告。

HDF 是硬件描述及导出产物，不等于完整 PetaLinux/BSP 工程，也不能据此推定 FSBL ELF 或 boot BIF 已经交付。当前最终 PL 在 Linux 启动后按初始化步骤动态加载，复现现有 PL 功能不要求重新生成 BOOT.BIN。HDF 后续加入不改变主实验 JSON 的历史时间或审批记录，只增加对当前交接文件与黄金 v5 关系的静态核对。

2.2.9 并列 STM32 LED 同步演示工程

src/stm32/gpio_pl_test_led 与原 gpio_pl_test 并列发布，说明名称为 GPIO_PL_Test_LED。为避免与黄金闭环工程混淆，该变体使用独立目录保存，不覆盖原成功工程；两份工程内部 Keil Target 均保留 GPIO_PL_Test，当前入口均为各自的 mdk-arm/gpio_pl_test.uvprojx，允许修改源码为 core/src/main.c。

该变体采用源码声明的 DK117E-G4 板载映射：PC8～PC15 经 M74LS573 驱动 LD1～LD8，低电平亮；PD2 保持 HIGH 使锁存输出透明跟随。主循环翻转 PA1 后立即调用 DK117E_LED_FollowPA1，读取 PA1 并更新八个用户灯，PA1 高时亮、低时灭。灯与 PA1 共用 HAL_Delay(500) 节拍，不另启 LED 定时器；存在软件读写延迟，不宣称零延迟电气同沿。PC8～PC15 与 LCD 数据线复用，本演示不同时运行 LCD。

表 2-5 并列 STM32 LED 同步演示工程

指示单元	所在硬件 / 用途	与判定的关系
四工具进度 LED	AX7Z020，AXI GPIO0x43C60000	四工具执行状态；不代表 DUT 物理达标
八用户 LED	STM32G431 / DK117E-G4，PC8～PC15＋PD2	随 PA1 输出相位的软件镜像，便于观察；周期仍由 PL 测量

上传变体实际源码已经使用 500 ms，源码 SHA 以 053697e2…7debfd 标识，完整值见附录 A。旧附件说明中 300 ms 基线没有沿用。新工程 92 份源码 / 工程 / 许可输入由 data/stm32_led_manifest.json 记录来源与双 SHA，生成产物和用户 IDE 选项排除。软件验证包含现有 Agent 对两工程入口 / 输出识别，以及提供主循环和 LED 函数的主机 HAL 替身；没有把历史 run_5d8e…或原 firmware.hex 作为 LED 变体的新实测 / 编译证据。八灯指的是用户灯，不包含电源、USB 或调试器指示灯。

2.3 软件系统介绍

2.3.1 软件整体架构

PC 软件由目标解析、Agent、Orchestrator、Controller、审批 / 范围绑定、四工具后端、观测适配、报告和 UI 构成。模型请求只接收任务、经批准源码与结构化证据；硬件命令由确定性后端构造。WorkflowEvents 统一供报告、UI 与 LED 订阅，不解析界面文本来猜阶段。API Key 在本机受控输入或进程环境提供，不写入公开配置及报告。

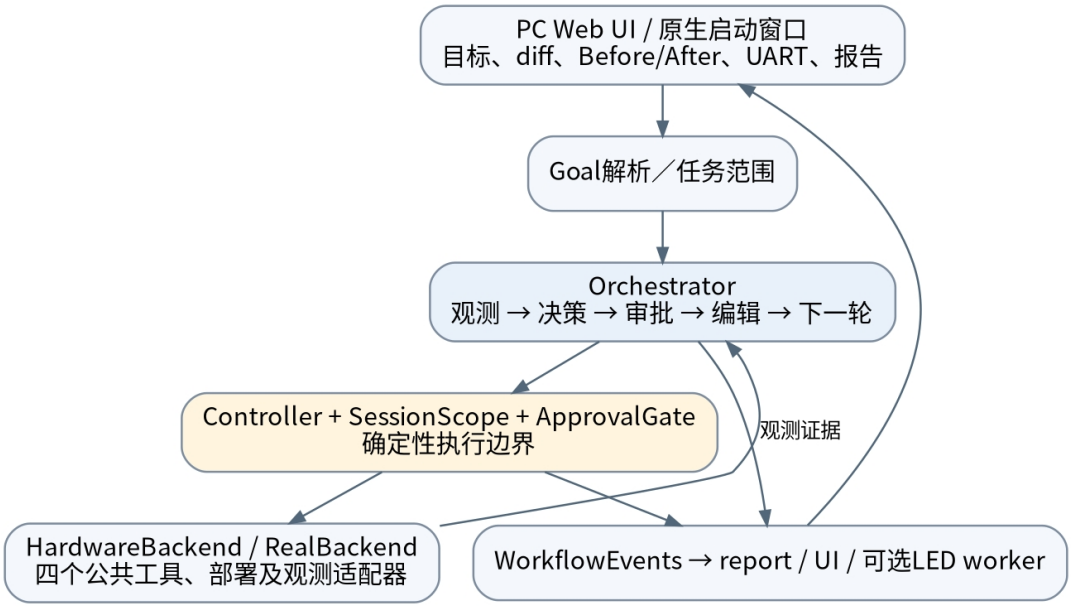

图 2-5 PC 软件模块与事件呈现的关系。

2.3.2 AI Agent / Orchestrator

目标解析把自然语言转为 test_id、observed_gpio、expected_period_ms 和 tolerance_ms；信息不足时澄清，不擅自扩大目标或修改限制。Orchestrator 每轮调用受控工具取得证据，先由确定性规则判断是否达标；未达标时再请求模型生成 edit 或 stop。edit 必须指向允许集合内的一个现有 UTF-8 文件，携带 before_sha256 与完整替换内容，不能包含新权限、命令或“模型自己宣布 PASS”的动作。

源码编辑获批后重新核对身份、应用并进入下一轮；本例最多 3 轮，实际 2 轮完成。每轮分别保留 execution、model_decision、applied_edit 等记录。任务停止、预算不足、工具失败或拒绝修改均有明确退出路径，取消后的硬件状态不确定性单独记录。

2.3.3 Reviewed Controller

Controller 限定 allowed_roots 及源文件范围、build→flash→log→test 顺序、单工具与全局剩余预算、最大 attempt 和停止规则；输入变化会使旧计划或旧审批失效。源码、固件、目标板、观测配置和会话范围相互绑定，避免把别的工程、旧 HEX 或不同部署条件的结果混用。审计保留每个工具输入、结果与错误码，模型无权批准自己的请求。

本例 UI 设置总预算 600 s；解析与组织会消耗时间，进入执行循环时传递剩

余预算而不是重置时钟。严格审批模式可逐轮确认；语义审批在确认会话范围后复用允许的执行边界，每个新 diff 仍需人工明确同意。

2.3.4 四个稳定工具接口

四个公共接口保持 contract_version=0.2。success 描述工具操作 / 采集是否成功，passed 描述确定性硬件目标是否满足；**success 不等于 passed**。第一轮 run_hardware_test 的 success=true、passed=false 正是这一语义区分的真实例子。

表 2-6 四个稳定工具接口

接口	主要输入	关键输出
build_firmware(project_path)	PC 工程路径	新产物路径 / SHA、编译日志、success
flash_firmware(firmware_path)	PC 本地固件绝对路径	远端 SHA、上传一致性、verify、工具日志
read_target_log(duration_s)	有限采集窗口	logs、健康字段、窗口及来源
run_hardware_test(test_id, params)	测试类型、目标与容差	measurements、observation、success、passed

2.3.4.1 build_firmware()

流程为检查工程范围、选择已配置 Keil target、执行 rebuild、解析日志并确认新 HEX，再计算 SHA。主输入是 PC 本地工程，输出不能被误写成 Zynq 路径。编译失败、输入身份改变或没有新产物时停止，不能拿上一次 HEX 继续烧录。最终两轮日志均为 0 Error(s)、0 Warning(s)。

2.3.4.2 flash_firmware()

面向目标 MCU 固件进行烧录，而非 Zynq/FPGA 配置。保持“输入 PC 本地固件绝对路径”的语义：本地校验与 SHA→SCP 上传至 /sd/firmware/<run_id>/firmware.hex→远端 SHA 一致性→SSH 执行 OpenOCD program / verify / reset→解析结构化结果。上传不一致或真实传输错误停止，不跳过完整性检查。硬件链为 Zynq USB→ST-Link/V2→SWD→STM32G431，不让模型拼接任意烧录命令。[11]

2.3.4.3 read_target_log()

RealBackend 默认绑定正式 PLUARTReader，确认独立 UART 配置与采集权

限，在一次 SSH 中完成 FIFO reset、有限窗口读取和文本组装。输出区分成功空日志、正常末尾半行和真正证据不完整。PC 侧超时、取消、SSH 失败以及健康错误均保留根因，不由 AI 修改源码掩盖访问故障。

2.3.4.4 run_hardware_test()

正式 PLPeriodReader 在已确认的部署上读取本次新边沿及一致快照，换算周期 / 高电平、频率和占空比，并按目标容差计算 passed。仅在观测本身成功后才比较数值，拒绝用旧寄存器或模型文字代替实测。该接口成功返回不等于目标通过，只有全部选定检查满足，外层才结束为 PASS。

2.3.5 RealBackend 与 Observation

RealBackend 封装 Keil、SSH / SCP、OpenOCD、受限 MMIO 脚本、PL 准备、周期读取与 UART 采集。实际 Linux 工具路径采用 /sd/tools/openocd/bin/openocd 及相应 scripts，使用 interface/stlink.cfg 与 target/stm32g4x.cfg；旧 Dropbear 兼容参数按已验证配置显式启用。对模型公开的硬件接口仍只有四个，部署检查、Linux 重封装和 HDF 检查属于操作 / 辅助入口，不增加第五个公开硬件工具。

当前部署顺序是 Linux 启动、网络 / SSH 就绪、部署者在初始化阶段手动加载 v5，再启动 Agent 和 LED；observer.prepare_policy 保持 check_only。check_only 只检查约定条件，不自动加载 PL，也不认证当前运行位流身份。代码保留受审批的 load_verified 可选策略，但本系统 BusyBox1.24.1 没有 timeout applet，不能未经依赖验证就切换。当前手动加载使用 cat 写/dev/xdevcfg，并检查 prog_done，具体步骤与失败后的核查见 board/zynq/pl_preparation.md。初始化加载和任务内 flash 后的内部检查是不同阶段；工具 / LED 默认执行顺序未因文档整理而改变。

2.3.6 人工审批与 Diff 安全机制

会话范围审阅目标、工程、允许文件、板卡、观测设置、最大轮次和预算；每次新源码提案展示完整 diff，绑定 before_sha256、proposal_id 和 fingerprint。批准前后若源码或范围变化，需要重新审阅。历史报告保留的指纹是审计证据，不恢复执行权限；断点恢复也必须重新确认当前目标与输入。

本例真实修改除 HAL_Delay(300)→500 外，还有一处 assert_failed 参数注释变动，报告保留完整差异，不能将其美化为“只改一行”。未来可进一步约束无关

改动，但本次不改黄金源码来重写已有证据。

2.3.7 PC 端可视化控制台

原生启动窗口选择部署 JSON、工程、源码和模型，API Key 遮罩输入；本机浏览器控制台展示目标、工具阶段、AI 工程理由、完整 diff、UART 轮次、Before / Target / After 及报告下载。普通用户无需先读大段 JSON，审计区仍保留完整数据供复核。真实模式、真实模型＋模拟硬件、离线演示明确区分。

当前固定提交另显示当前运行报告 ID 与历史所选 ID：轮次表、当前下载及审计 JSON 对应当前运行；历史选择只决定下载 / 恢复入口，不切换当前结果。原实测截图记录的是此前界面，保留原样，不伪装成更新后重新执行的硬件测试。

2.3.8 LED workflow 状态同步

可选 status_indicator 配置随 SessionScope 绑定，LED adapter 消费事件的 kind、tool_name、attempt 与 TOOL_FINISHED.success。后台 worker 只写固定地址和 4-bit 值，负责普通闪烁、双闪及保持状态；新轮 BUILDING 清空上一轮，四工具成功后保持完成状态。等待 AI 或 diff 审批期间维持本轮已完成状态。标准四接口全部成功后四灯保持完成；显式 PL-only 跳过 UART 时 LED3 保持灭，即使被选物理检查 PASS 也不伪装成四工具都执行。worker 启动、SSH 或 LED 写入失败只形成独立警告，不能覆盖四工具的 success 或 DUT 的 passed。

2.3.9 Linux 基础系统与 rootfs 定制部署

2.3.9.1 来源与版本

本文采用队伍实际完成的工程路线：ALINX 官方 / 配套 AX7Z020 预构建 Linux 作为基底，验证板卡启动后定制 ramdisk/rootfs 网络配置，再封装 FIT 并部署。基础系统由合法配套介质取得，厂商入口见参考文献[13]；精确历史下载包名 / 直链未在已核对材料中给出，收到镜像以 SHA 锁定。当前版本依据队伍终端截图及原 ramdisk 的/etc/issue，记录在 data/linux_platform.json，不将内核构建日期当作镜像发行版本。

表 2-7 来源与版本

项目	已确认值 / 依据
Target OS	PetaLinux2017.4；/etc/issue 与终端截图相符
Linux / 架构	4.9.0-xilinx；armv7l / ARM32

项目	已确认值 / 依据
BusyBox	v1.24.1
内核工具链	Linaro GCC6.2.1（2016.11）；gcc6.2.1 20161016
内核构建时间	2019-10-18 17:53:51 CST；供应商内核时间

截图中/etc/os-release 不存在，版本核对采用 uname、/proc/version、BusyBox 和/etc/issue。现有材料包括 BOOT.BIN、image.ub、image_fixed.ub、当前 HDF、v5 位流和 ARM OpenOCD 工具 / 源码；队伍另确认有 image.ub.known_good 备份，但已核验的两分包未包含该备份，其 SHA 保留待本地单独登记。两分包原名 Zynq_Linux_Test.zip，确实包含 SD 启动 / 部署材料，不能因目录名不同认定未提供。

2.3.9.2 rootfs 网络修改与 FIT 重封装

实际改动是将 etc/network/interfaces 中的 eth0 由 DHCP 改为静态 192.168.10.2、netmask255.255.255.0，保留其他接口。原始 image.ub 与历史 image_fixed.ub 的 kernel 及 FDT 载荷 SHA 相同，变化在 ramdisk。历史 fixed 还增加了设备专用 Dropbear 主机私钥，并调整 vt200 / vt220 的文件放置；公开说明如实保留这些差异，新的可复现方法只重现网络改动，不复制现场私钥或无关终端改动，所以不宣称新产物与历史 fixed 整体 SHA 相同。[14-15]

build/linux/customize_rootfs.py 先验证已知基底 SHA、FIT 三个载荷的 hash 及默认启动配置，取出 ramdisk@0 中的 gzip newc CPIO。脚本在 CPIO 记录层替换唯一网络文件，原样保留其他文件、设备节点、uid / gid、模式、符号链接和硬链接字段，无需向主机文件系统解包 rootfs。随后 gzip 压缩、生成引用原 kernel / FDT 与新 ramdisk 的 image.its，调用 mkimage 重新封装并更新 hash；保留内核 load / entry 和配置元数据，最后重新解析核对。

收到镜像的 FIT ramdisk 节点 compression 为 none，而 data 本身是 gzip CPIO，脚本沿用该层级。支持范围限定为已收 inline-data FIT 格式；未知基底、签名 FIT、外置 data、坏 hash、不安全 CPIO 名称和已有输出目录会拒绝。每次产生独立工作目录和 build_result.json，明确 boot_bin_rebuilt=false、board_tested=false。该辅助脚本只处理本地文件，不经模型调用，也不连接板卡或写 SD。

2.3.9.3 SD 部署、SSH 与 v5 初始化

SD 根目录沿用 BOOT.BIN 及实际被 U-Boot 加载的 image.ub，另保留本机回退副本；tools/openocd、pl/zhidiao_observe_v5_led.bit 和 firmware/<run_id>按既有契约部署。只把 image_fixed.ub 放在旁边不代表启动器会加载它，选定产物需按当前契约安装为 image.ub。备用卡分区和启动开关遵循 AX7Z020 配套资料，本文不编造未提供的扇区参数。v5 在 Linux 启动后从 /sd/pl/zhidiao_observe_v5_led.bit 写入/dev/xdevcfg，不要求重新生成 BOOT.BIN。

按 board/zynq/pl_preparation.md 设置 PYTHONPATH、恢复并验证 SSH 公钥登录、核对 v5 文件 SHA、手动加载及确认 prog_done，再启动 Agent 和 LED。上位机保持严格 known_hosts 及 BatchMode 公钥登录要求。原基底 Dropbear 启动脚本在缺主机密钥时会生成，但这不保证跨 initramfs 冷启动持久化；每块板使用自己的身份并通过可信通道核对指纹。SSH 连接、用户公钥恢复及主机密钥身份分别记录，不把密码登录截图当作免密持久化证明。

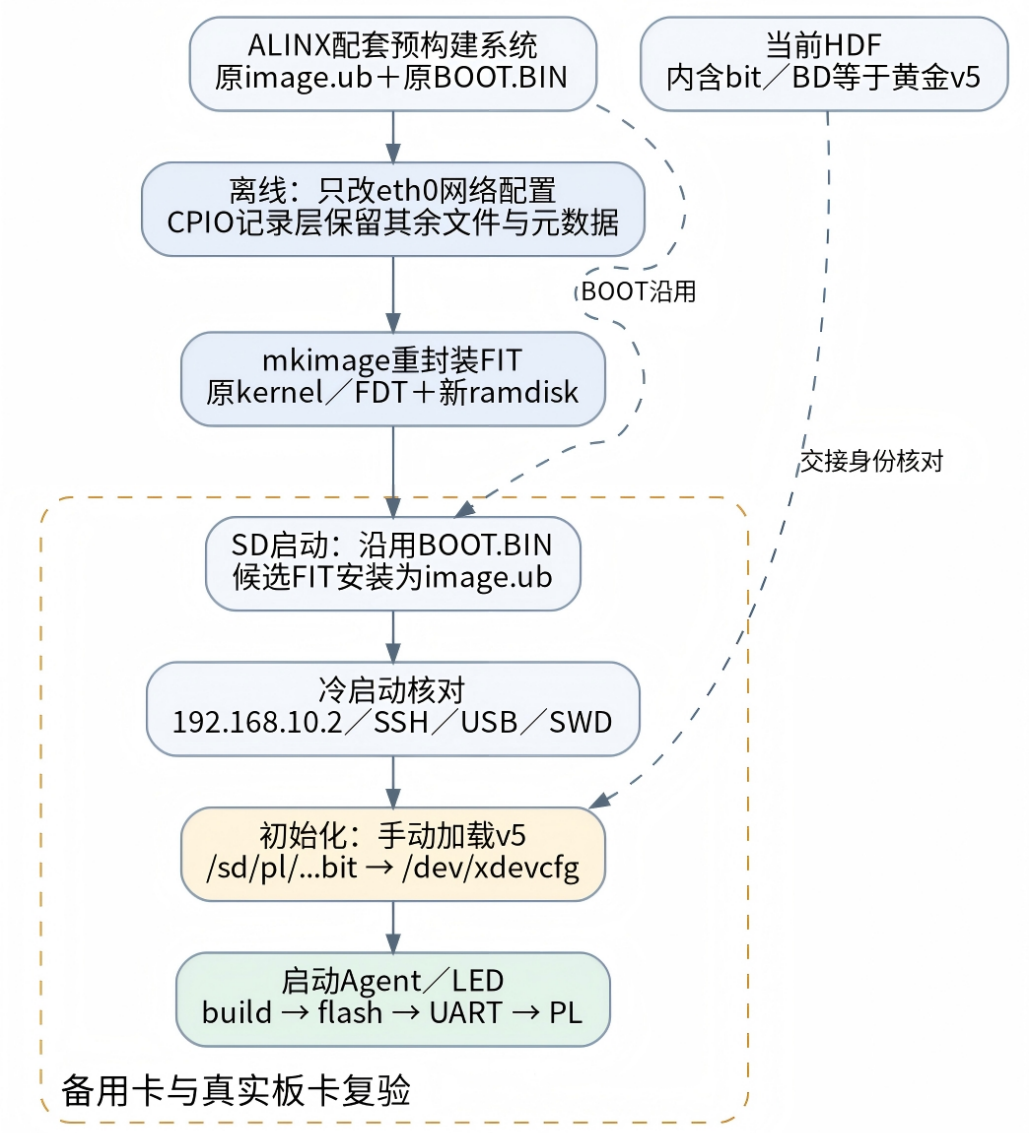

图 2-6 Linux 预构建镜像定制、部署与 v5 初始化。

2.3.9.4 实现与验收范围

当前没有原始完整 PetaLinux/BSP 工程、kernel / U-Boot / 设备树源码、完整 rootfs 源码配方、FSBL ELF 或 boot BIF，因此本报告不声称执行 petalinux-build 或从源码重建整套 Linux，也不把补做这些项目列为当前路线的必补任务。交付重点为基底来源 / 版本和 SHA、实际 rootfs 网络补丁、FIT 重封装、SD 部署及冷启动复验方法。

原系统冷启动网络 / SSH、USB Host 和 OpenOCD / ST-Link / SWD 链路已由队伍验证；本次终端截图确认运行版本。离线新生成镜像仍标记 board_tested=false，需在备用卡独立完成启动与链路复验。

第三部分　完成情况及性能参数

3.1 整体介绍

已实现的系统由 PC 控制台、Zynq PS / PL 平台、ST-Link 和 STM32 目标板组成，形成真实构建、部署、日志、测量及源码重试链路。最终主证据是 run_5d8e004954664f6c8f8ff0f85d31959b 及对应源码、HEX、v5 位流和网页截图。多张截图和不同导出快照仍属于同一次运行，不能重复统计为多次成功。

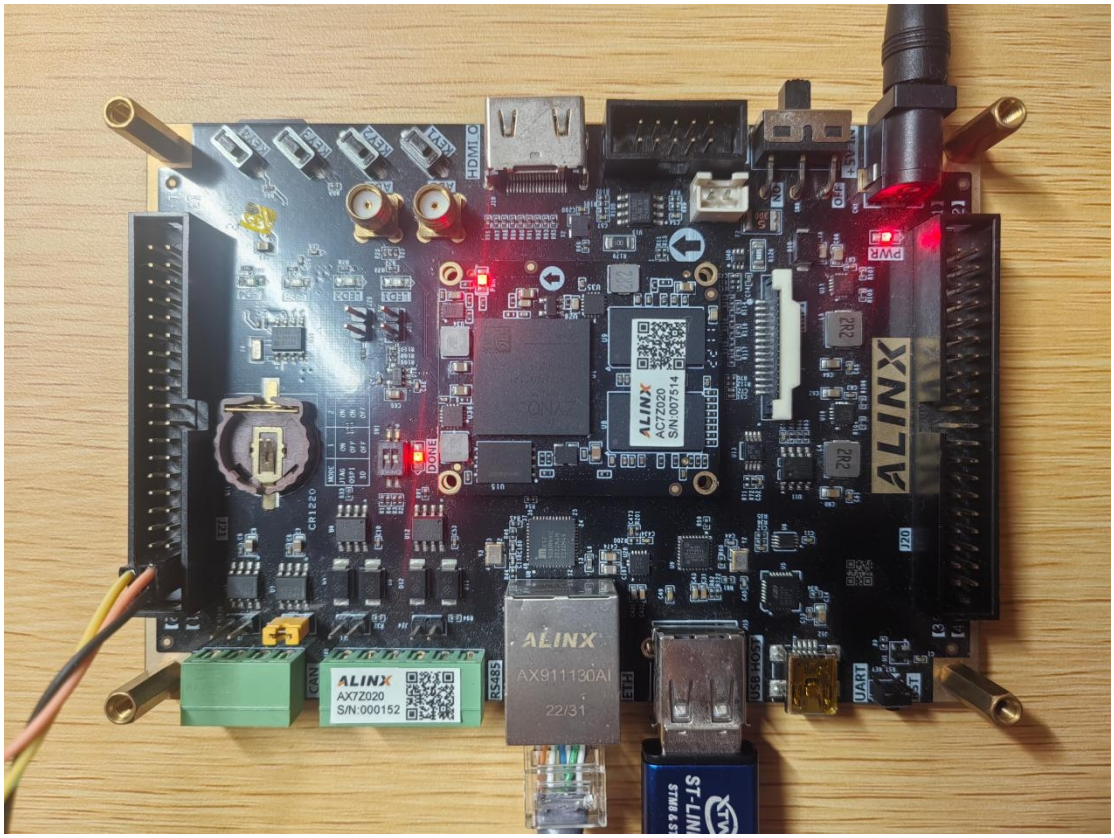

图 3-1 开发板正面实物预览。采用 AX7Z020 Zynq7000。

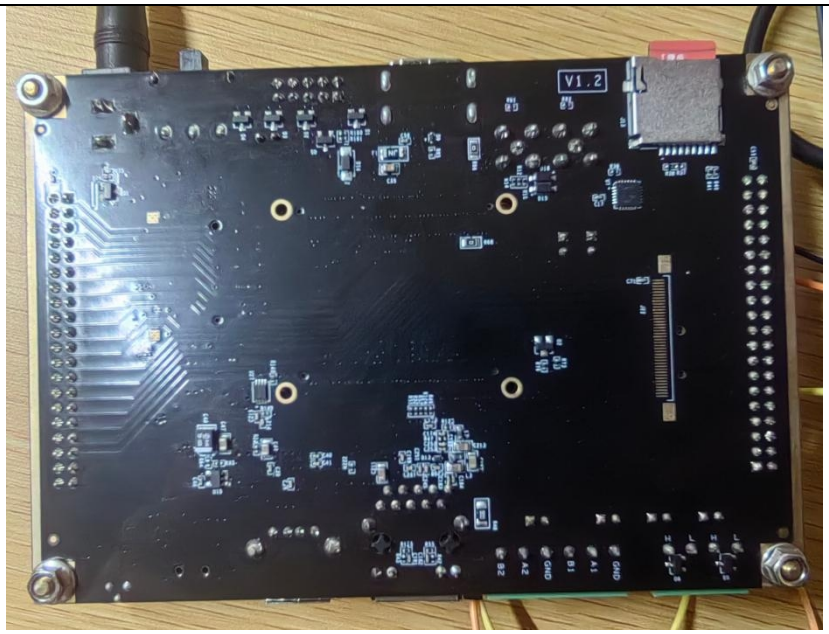

图 3-2 开发板背面实物预览。采用 AX7Z020 Zynq7000。

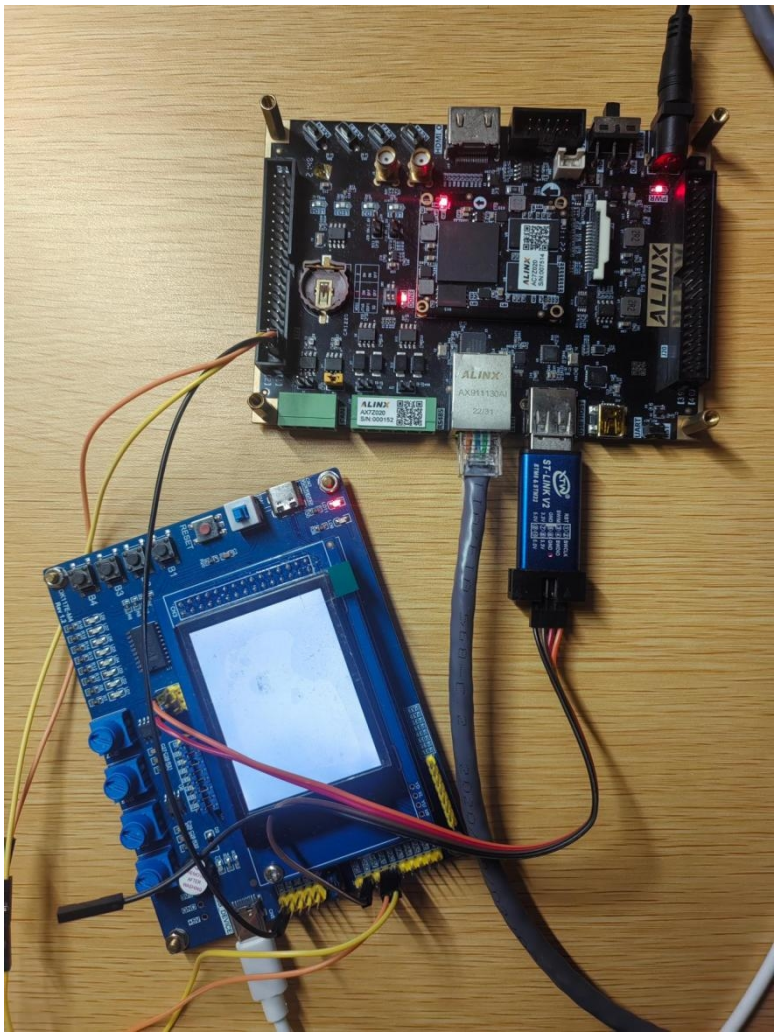

图 3-3 智调盒真实硬件测试平台。系统由 Zynq-7000、STM32G431RBT6、ST-Link 及 GPIO/UART 观测连线组成。

3.2 工程成果

3.2.1 硬件与 PL 成果

整理出的最终工程包含原创测量 RTL、BD、导出 Tcl、wrapper、四份 XDC、XPR、当前 HDF、v5 位流及 baseline / final 资源与时序报告。BD 实现周期 / 状态 AXI 寄存器、UART 和 LED，同时保留供应商摄像头 / HDMI 底座，因此全工程资源不是测量核的单独资源。合法外部 IP 以 VLNV 和 37 份文件 SHA 清单交付，不把未附完整再分发许可的目录整体公开。

本版没有独立机械加工或自研 PCB 资料，不将通用开发板搭建描述为已完成外壳产品。

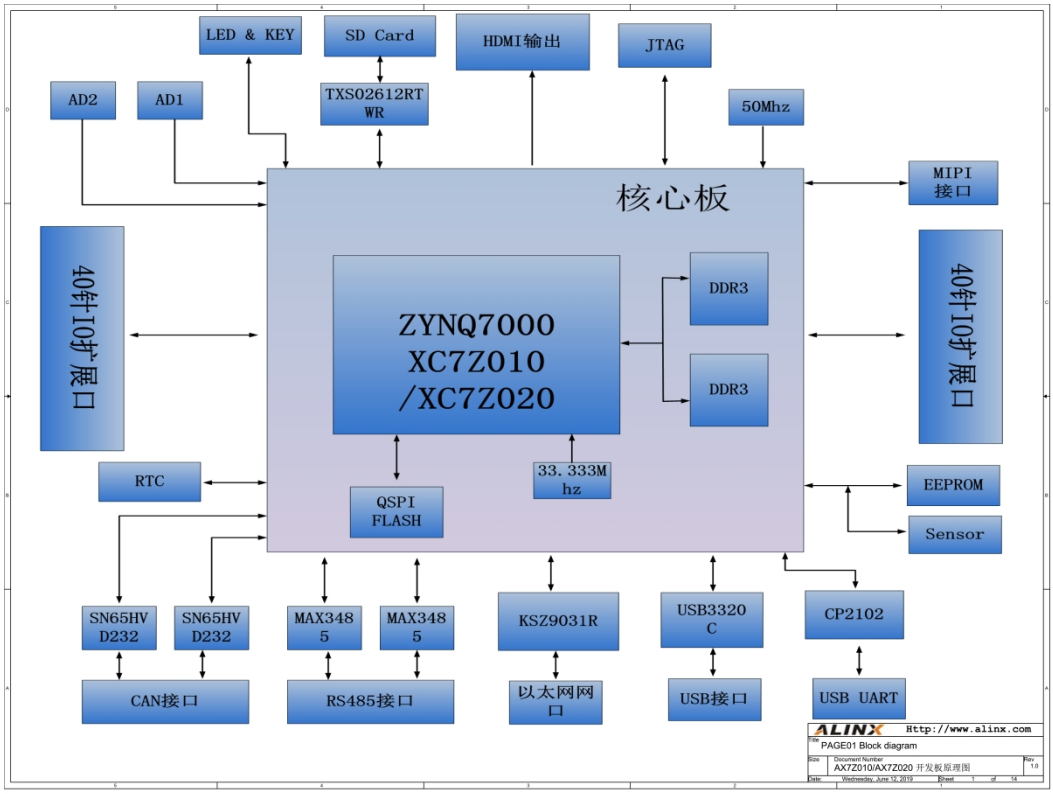

图 3-4 AX7Z020 开发板硬件布局原理图框图参考。

3.2.2 电路 / 接口成果

已核对 M18、M17 及 LED J14 / K14 / J18 / H18 的原始 XDC，与当前部署地址和实验接线一致，详见图 3-3。STM32 原工程保留 CubeMX、core、drivers、Keil Target 及启动输入；黄金 core/src/main.c 为最终已通过内容。LED 同步工程单独并列，源码与产物身份不混用。板端路径约定、OpenOCD 源码及合法构建

入口、文件检查脚本和 PL 准备说明均已整理，原 SSH 主机私钥及含密钥镜像没有公开。

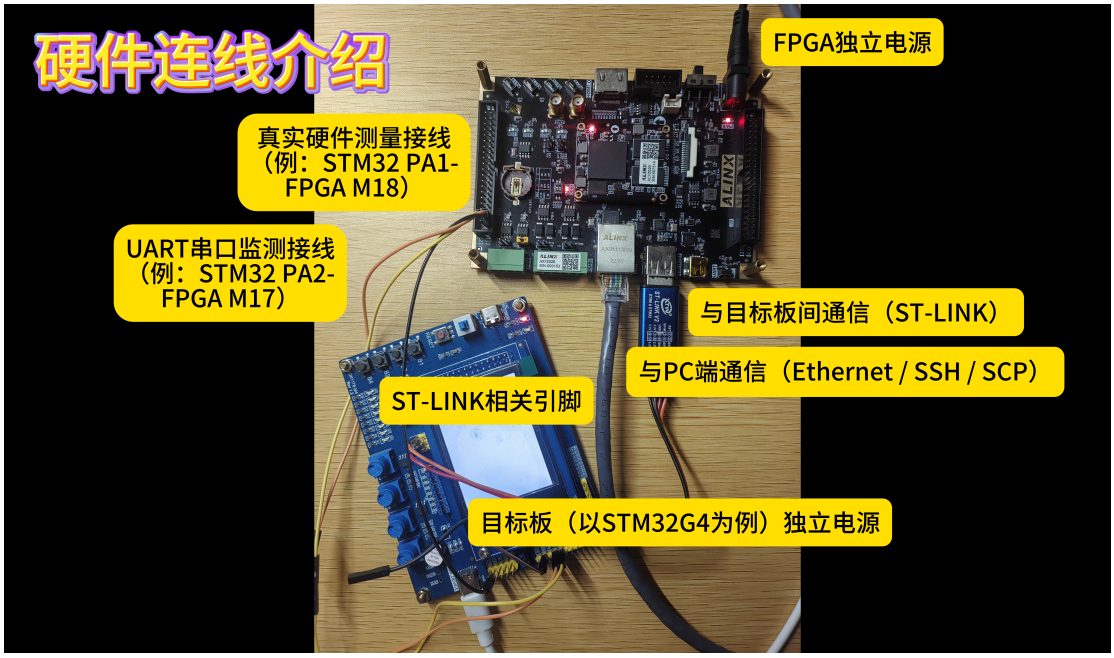

图 3-5 智调盒真实硬件测试平台连线介绍。

3.2.3 软件成果

PC 控制台能够组织自然语言目标、范围审批、源码 diff 审批、四工具阶段与最终结果。图 3-6 显示主实验两轮结果；原图中的工程路径为测试环境信息，数值未重绘。独立 JSON 保存模型理由、实际源码提案、审批、两轮工具结果与 freshness，可与文件 SHA 交叉核对。[14]

图 3-6 主实验原始网页界面：601.717 ms FAIL，经修改后 1001.087 ms PASS。

图 3-7 中可见第二轮两条 ZHIDIAO_OK 与轮次表。UART 日志说明本次确有软件输出，周期仍由 PL 计量，不由日志字符串宣布达标。

图 3-7 同一主实验的 UART 及轮次结果原图；不是另一次独立实验。

3.2.4 交付目录、命名与输入身份

当前仓库按 src、sim、build、board、data、skill、report 映射 AMD 推荐目录。发布路径统一英文小写、数字、下划线或连字符，扩展名使用点号；保留指南目录示例中的根 README.md 及 skill/README.md。源码变量、Keil Target、外部

SDK 逻辑名称及板端 BOOT.BIN 约定文件名不属于仓库文件重命名。原始第三方源码包、HDF 内部名称和版权条款保持上游原字节。

表 3-1 交付目录、命名与输入身份

内容	当前入口	核验与范围
原 STM32 工程	src/stm32/gpio_pl_test/mdk-arm/gpio_pl_test.uvprojx	对应黄金 main 及原两轮报告；新构建输出在本工程 mdk-arm/gpio_pl_test/
LED 同步变体	src/stm32/gpio_pl_test_led/mdk-arm/gpio_pl_test.uvprojx	八用户 LED 辅助演示；独立源码及离线/主机替身验证，不作为主闭环真机证据
FPGA 与交接	src/fpga/zhidiao_measure/	原输入 14 项清单与新 HDF 分别登记；HDF 内 bit / BD 等于 v5
Linux 定制	build/linux/customize_rootfs.py	固定原 image.ub 基底，只改网络；不重建 BOOT
初始化 / SD	board/zynq/pl_preparation.md、sd_deployment.md	当前手动预加载 v5 与 check_only 顺序
发布检查	sim/check_release.py	原 / LED 工程身份、HDF、原报告和原图等只读核对

data/repository_path_map.json 记录 123 份改名文件的旧 / 新路径与旧 / 新 SHA，并保护 24 份历史证据。旧 JSON、完整 diff 中的 Core/Src/main.c 保持当时记录；当前文件为 core/src/main.c，黄金 main 的字节和 SHA 不变。两份项目中只有本地引用 / 名称需要适配，原 Keil Target 仍为 GPIO_PL_Test。启动器优先建议小写路径并兼容已有旧目录，私人部署配置由用户按当前工程重新预览和批准，不自动覆盖旧配置或重放历史权限。

3.3 特性成果

3.3.1 四个公开接口真机验证

表 3-2 四个公开接口真机验证

接口	主实验第一轮	主实验第二轮	证据解释
build_firmware	success=true；0 error / warning	success=true；0 error / warning	均产生新 HEX，记录 SHA

接口	主实验第一轮	主实验第二轮	证据解释
flash_firmware	upload_verified、verified 均 true	upload_verified、verified 均 true	本地 / 远端 SHA 一致，含 program / verify / reset
read_target_log	3 完整行＋末行半行	2 条完整日志	健康字段无错；不是跳过 UART
run_hardware_test	success=true，passed=false	success=true，passed=true	两轮 fresh_edges=2，观测有效

这里的“真实验证”依据已提供原始运行记录，不是本次文件制作时重新连接硬件。两轮四工具执行顺序均完整，第一轮物理 FAIL 没有被误写成接口调用失败，第二轮才满足目标容差。

3.3.2 完整 AI 调试闭环实验

用户目标是“把 STM32 PA1 的完整周期调到 1000 ms，误差不超过 50 ms”。第一轮在 HAL_Delay(300)程序上测得周期 601.7169 ms、高电平 300.36864 ms；模型识别一个完整周期包含两次翻转等待，提出 500 ms 单次延时。人工批准提案后重新构建目标 MCU 工程，并通过 Zynq→ST-Link→SWD 重新烧录 STM32G431；Zynq/FPGA 继续作为既有执行与 UART、PL 观测基础设施。第二轮周期 1001.08682 ms、高电平 500.07403 ms，绝对误差 1.08682 ms，确定性判 PASS。

表 3-3 完整 AI 调试闭环实验

量	第一轮	第二轮
period_ticks	60171690	100108682
high_ticks	30036864	50007403
完整周期 / ms	601.7169	1001.08682
高电平 / ms	300.36864	500.07403
绝对误差 / ms	398.2831	1.08682
新边沿数	2	2
DUT 判定	FAIL	PASS

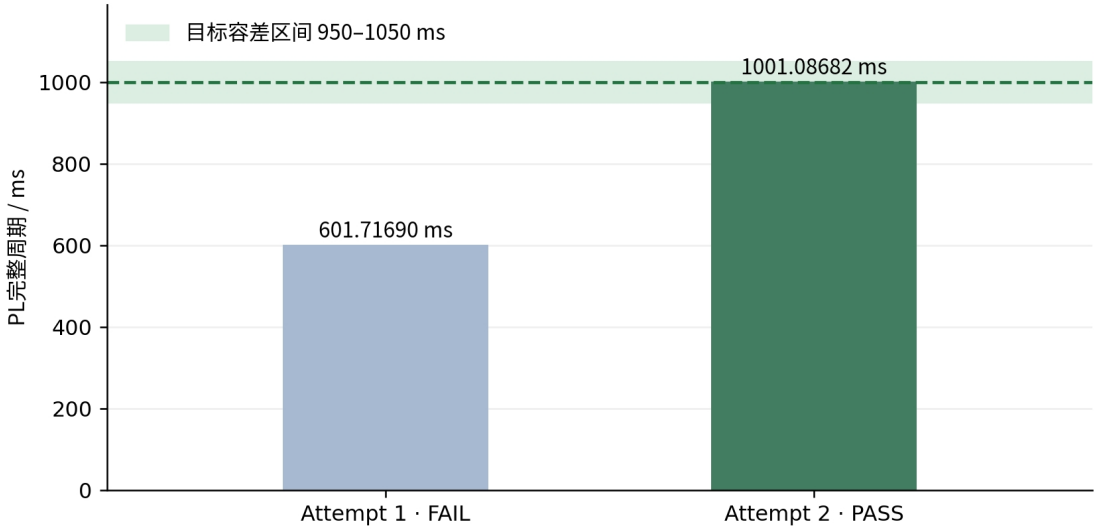

图 3-8 根据主 JSON 绘制的两轮周期对照，绿色区域为目标容差；不是仪器截图或多次样本分布。

完整差异保存于 data/final_source_change.diff。核心改动如下，同时保留原提案的一处注释变化：

```
- HAL_Delay(300);
+ HAL_Delay(500);
- * @param  line: assert_param error line source number
+ * @param  line: assert_param line source number
```

网页显示三位小数 601.717 / 1001.087 ms，对应 JSON 原值；不是两组不同测量。最新网页导出总 elapsed_s 为 81.18937509999523，与旧归档 81.16940819999581 只差约 0.0199669 s；其余原始 JSON 字段一致，仍计为一次运行、两轮 attempt。

另有图 3-9 的 1000.192 ms 单轮基线场景，界面明确跳过 UART、无需 AI 修改。该场景本次只有截图，没有独立 JSON 或可读 run_id，单独作为 PL-only 参考，不合并到主实验的四接口结果。

图 3-9 补充 PL-only 基线已达标原图；没有 UART 采集，也没有 FAIL 后修改过程。

硬件部分，在完整闭环执行过程中，Zynq PL 侧的四路板载 LED 同步显示四个公开工具的执行进度。四灯依次对应 build_firmware、flash_firmware、read_target_log 和 run_hardware_test；随着构建、目标 MCU 烧录、UART 采集和 PL 硬件测试依次完成，相应 LED 从执行状态（闪烁）进入完成状态（常亮），并在本轮任务中保持，从而使软件工作流在真实硬件上获得直观的状态映射。

图 3-10 到图 3-14 中连续实物帧展示了闭环执行过程中 LED 状态随工具阶段推进而变化的实际效果。该 LED 仅用于表示工具工作流进度，不参与 DUT 的物理结果判定：例如第一轮 run_hardware_test 虽然工具执行成功，但实际周期为 601.7169 ms，仍由确定性规则判定为 FAIL；第二轮测得 1001.08682 ms 后才判定 PASS。**因此，LED 所呈现的是“工具是否完成”，而非由灯光替代 FPGA 物理测量结果。**

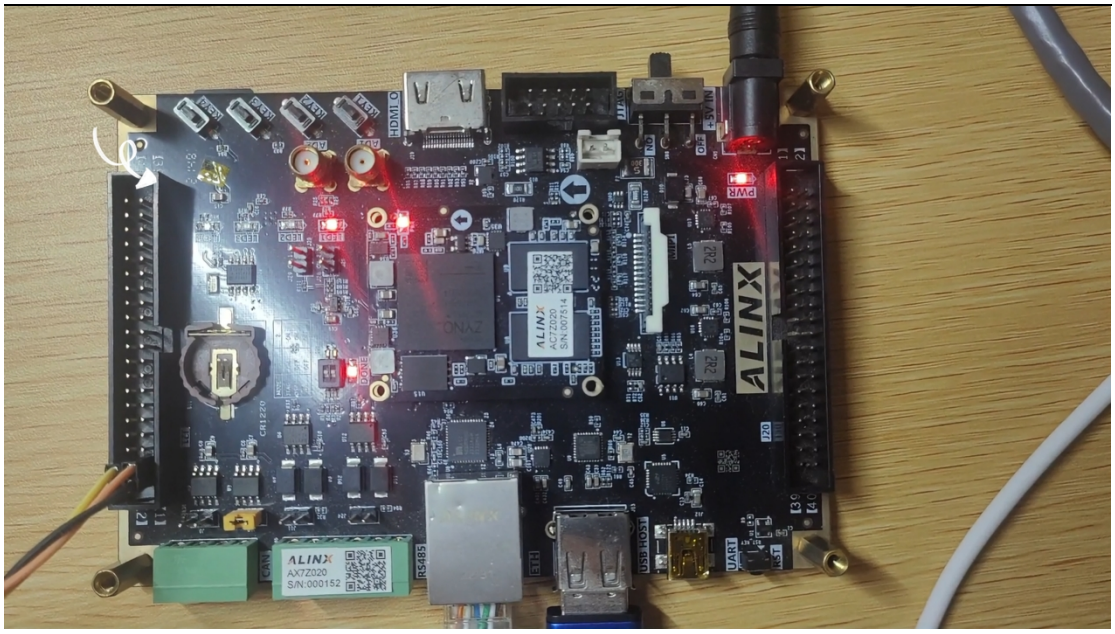

图 3-10 LED 1 亮起，程序编译构建完成

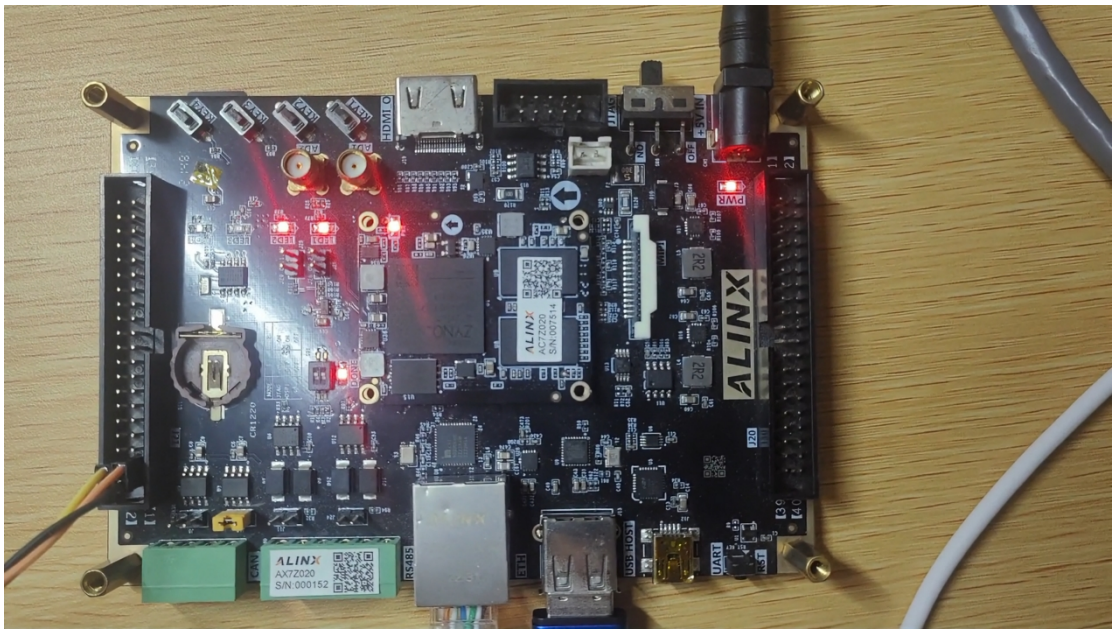

图 3-11 LED 2 亮起，目标 MCU 烧录完成

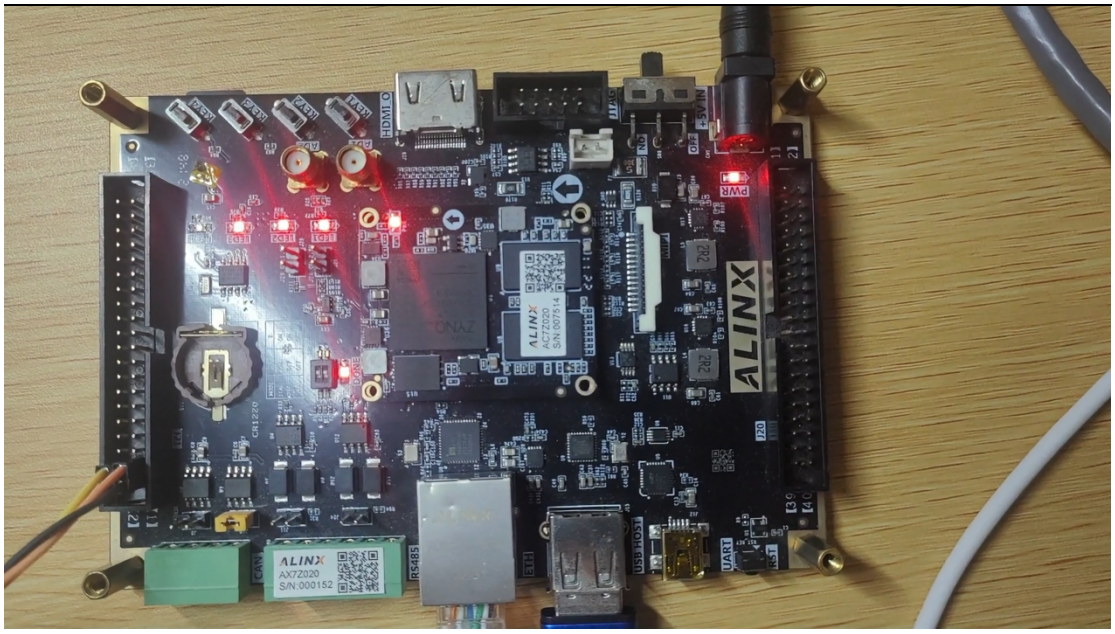

图 3-12 LED 3 亮起，UART 采集完成

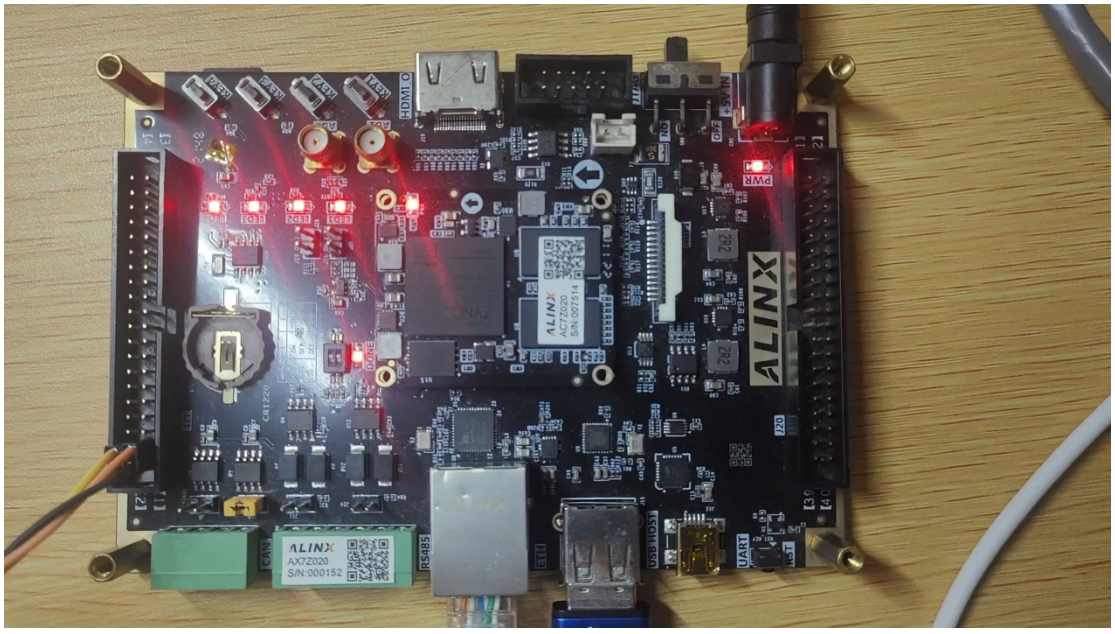

图 3-13 LED 4 亮起，PL 硬件测试完成

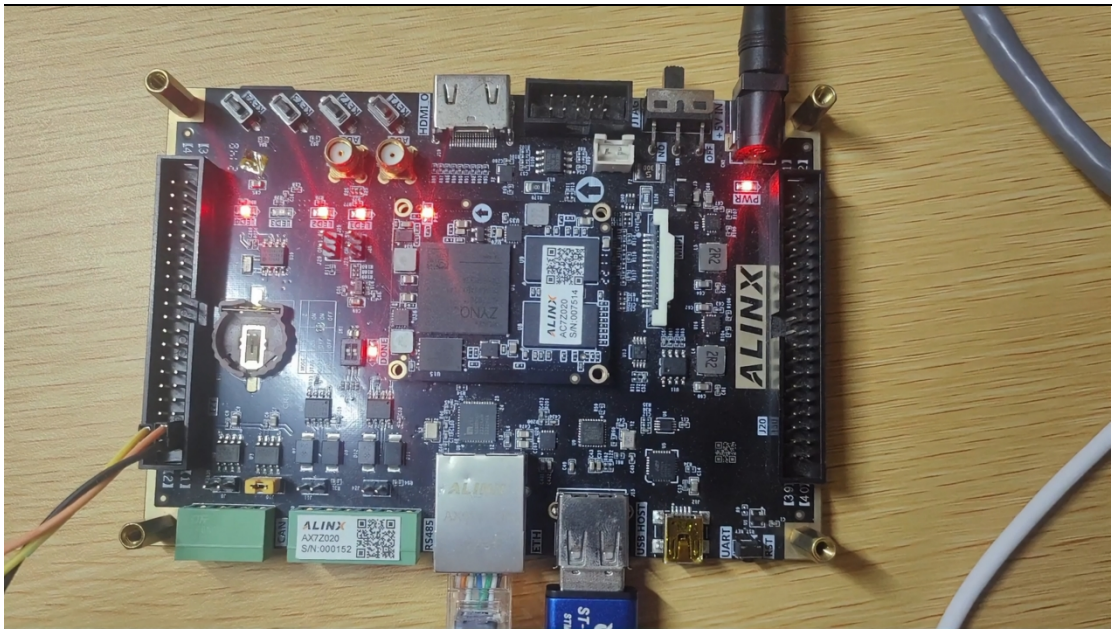

图 3-14 在初始化页面关闭“采集 UART”选项时，LED 3 不亮

3.3.3 功能正确性与异常验证

验证按真实记录、离线测试和已知边界分层，不能把 mock 或单元测试写成已做过对应真机负向实验。现有主 JSON 证明完整正向闭环和正常 UART 半行处理；其他故障类型主要由离线回归覆盖。

表 3-4 功能正确性与异常验证

场景	规则 / 预期	已有证据层次
本轮无新边沿 / 陈旧寄存器	freshness 不足，不判 PASS	PL reader 离线回归；主实验有新边沿正例
成功但没有串口文本	success=true、logs=[]可继续 PL	UART / workflow 离线回归；不代替 GPIO 证据
deadline 末行半行	partial_tail=true、truncated=false 继续	主实验第一轮真实发生并继续 PL
capture_limit 或健康错误	保留致命不完整 / 故障停止	UART 窗口边界及 Controller 回归
SSH / 传输失败	保留连接或传输根因，停止	后端与采集离线回归
人工拒绝 diff / 取消	不继续未经批准修改；记录停止	审批、取消和报告回归；原错误资料有取消记录
源码 / 范围外部变化	旧指纹失效，重新确认	Session / editor / workflow 回归
LED worker 异常	记录 indicator warning，不覆盖 DUT	LED 状态适配器离线专项

场景	规则 / 预期	已有证据层次
输入 HIGH 启动首次结果	可能含启动残段，需要后续稳定周期	原 RTL 实际 Icarus 仿真，边界尚未修改

当前固定提交对应已归档的 Linux Python3.13.5 Agent465 项、根 sim32 项结果，均 OK、无 skip；分别记录 23.968 s 与 2.066 s，来自不同范围的实际执行，不把历史计数相加。Agent 生产代码及测试在最新 Linux / HDF 补充中未改，465 项沿用对应源码结果；32 项已在相同 Git 树的干净导出副本执行，包括 Icarus 实际 RTL、LED 主机 HAL 替身、42 个 C 单元预处理、实际 mkimage 及反向篡改检查。依据为 data/test_summary.md 、 path_migration_verification.txt 和 linux_customization_verification.txt。

图 3-15 中历史下拉选中 run_2178…，展开 JSON 却为主案例 run_5d8e…，源于两个字段用途不同。最新代码已加分别标识和说明，并用实际 JavaScript 脚本回归确认选择历史不会改变当前测量 / JSON；原图保持原样。独立文件检查也明确输出 hardware_result=NOT_TESTED，不把文件存在当作硬件通过。

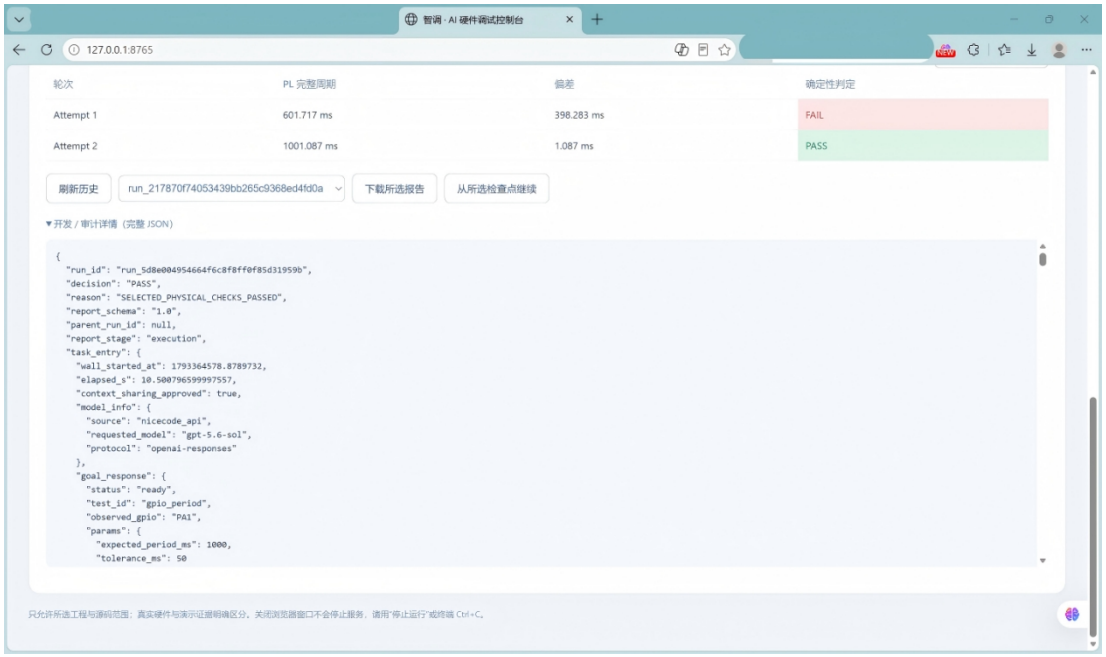

图 3-15 原始审计界面：历史选择用于下载 / 恢复，展开区对应当前报告。

3.3.4 性能与资源指标

下表取自原始 placed utilization 和 routed timing summary，已由脚本核对。基线为供应商摄像头 / HDMI 参考工程，final 是加入观测、UART 与 LED 后的整

套设计；不是严格的 v4→v5 同功能优化对照，也不能把全工程资源归因于测量核本身。

表 3-5 性能与资源指标

指标	供应商基线	最终观测工程	变化 / 说明
LUT	6756 / 53200（12.70%）	7274 / 53200（13.67%）	+518，约+7.67%
FF	10853 / 106400（10.20%）	12083 / 106400（11.36%）	+1230，约+11.33%
BRAM Tile	20.5 / 140（14.64%）	20.5 / 140（14.64%）	0
DSP	0 / 220	0 / 220	0
IOB	38 / 125（30.40%）	44 / 125（35.20%）	+6
WNS / ns	1.091	0.709	已指定约束的报告值
TNS / ns	0	0	无已报告的总负裕量
WHS / ns	0.047	0.020	hold 最坏裕量
主观测时钟	100 MHz	100 MHz	名义测量时钟

报告写明所有已指定时序约束满足，但同时保留 HIGH 警告，已按实际使用通路补约束或说明合理豁免。[3-5,14]

表 3-6 性能与资源指标

主 JSON 计时项	第一轮 / s	第二轮 / s	口径
构建接口 duration	11.593	11.474	PC 包装的工具耗时；编译器日志另记 6s
烧录接口 duration	2.856	4.110	含传输、校验和远程工具
UART 接口 duration	5.569	6.558	含 SSH 与采集开销
UART 实际窗口	5.000	5.020	capture_elapsed_ms 字段
PL 测试接口 duration	1.021	1.472	本次有新边沿时的观测耗时
四工具 duration 求和	21.039	23.614	不是独立测得的全局延迟

主报告 elapsed_s 约 81.189 s，仅代表此次运行计时口径；不能把包含模型、审批和组织开销的值当成纯硬件耗时或平均吞吐率。当前只有主案例两轮数据，不据此给出成功率、平均收敛时间或效率提升统计。

3.3.5 优化过程及前后对比

表 3-7 优化过程及前后对比

原问题 / 早期方案	形成的规则 / 实现	已确认改善与边界
直接读寄存器可能复用旧值	新边沿、valid 和一致快照	建立 freshness；不是所有启动相位安全证明
老 BusyBox 不支持小数 sleep	整数微秒 usleep 及有限预算	与既有板端环境兼容，保留超时
UART smoke / 临时注入 reader	正式 PLUARTReader 与独立配置	自动接入后端，来源、窗口、健康可审计
正常末行被当成 truncated	partial_tail 与 capture_limit 分离	真实半行后 PL 继续，真错误仍严格停止
每个动作反复确认	会话范围＋每个新 diff 审批	审阅层级更合理，身份与次数约束不省略
终端 JSON 为主要展示	Web UI、三段数值和报告下载	面向用户呈现流程，审计仍可查看
工具状态仅在终端	可选 4-LED 事件适配	已有设计与状态映射验证；LED 用于显示工具执行状态，不参与 DUT PASS/FAIL 判定
UI 真实模型隐藏 30s 限制	NiceCodeConfig timeout_s=120	补丁随最终实测代码存在；仍受总预算约束

这些优化有的是兼容性、可信性或使用流程改进，不全部对应资源降低。资源表描述增加能力的代价；目标周期误差从 398.2831 ms 收敛到 1.08682 ms 是本次任务结果，不能替代同功能资源压缩或多样本性能实验。

3.3.6 大模型协作记录

本项目中的大语言模型（Large Language Model, LLM）不仅用于开发过程中的方案讨论，也被实际接入智调盒运行时闭环：模型根据用户目标、经批准的源码以及构建、烧录、UART 和 PL 返回的结构化证据进行分析，并在目标未达

标时提出源码修改或停止建议。工具执行顺序、权限、预算、人工审批及最终 PASS / FAIL 判定仍由确定性 Controller、人工审阅和物理测量共同约束，LLM 不直接获得任意硬件命令执行权。

本节按照“LLM 参与了什么—实际采纳了什么—哪些路线未采用—如何提炼为可复用经验”的顺序，结合已留存的提示模板、模型回答、工作流、源码 diff 和重新验证结果说明协作过程。交互材料用于支持工程判断，不以聊天数量或未保存的对话替代可核验记录。

3.3.6.1 LLM 在闭环中的作用与人工分工

系统围绕“用户描述目标、真实证据驱动下一步”的思路进行分工。LLM 通过 Agent / Orchestrator 接收自然语言目标、允许范围内的源码和本轮结构化证据，负责解释观测结果、形成工程推理并提出 edit 或 stop 建议；Controller 负责工程范围、工具顺序、时间预算、最大轮次、审批状态与停止条件；四个稳定工具负责真实构建、烧录、UART 采集和 PL 物理测量；队伍成员负责接线与部署确认、完整 diff 审阅以及最终工程验收。

这种分工有意将“推理能力”和“执行权限”分离。LLM 可以回答“根据当前证据，下一步值得尝试什么”，但不能自行批准修改、拼接任意硬件命令或宣布目标已经通过。**模型提出的解释只有在人工批准并经过新一轮真实硬件测量后，才能转化为已验证结论。**

根据设计资料与实现归纳，实现入口为 agent.py、orchestrator.py、controller.py 及 tools 目录。

失败诊断中，原错误记录出现 read_target_log success=true、健康错误为 0，但末尾仅有 Z / ZH、truncated=true，Controller 以 LOG_INCOMPLETE 停止。工程分析区分正常采集窗口结束的半行与真实证据丢失，形成 partial_tail 字段和回归；最终主实验第一轮的 ZHIDIAO_O 及后续 PL 结果支持修复有效。故障事实来自原日志，修复过程按代码和设计资料归纳。

3.3.6.2 已采纳建议、人工判断与验证

案例二有主 JSON 直接证据。原文为“把 STM32 PA1 的完整周期调到 1000 ms，误差不超过 50 ms”。LLM 的输入包括任务、实测证据和批准的源码。以下为现有源码中的系统提示模板原文摘录：

Propose changes and explain evidence. Never claim that tools ran or tests passed without supplied results. You cannot approve or execute operations.

The deterministic controller owns tool order and retry limits. Distinguish tool success, flash verification and hardware test pass.

LLM 实际返回 action=edit，目标文件在历史 JSON 中记录为 Core/Src/main.c，before_sha256 与该轮原文件绑定。该路径保留原貌，当前仓库对应 core/src/main.c，见路径映射。reason 字段原文如下：

真实 PL 测得 PA1 完整周期为 601.7169 ms，高电平为 300.36864 ms，且当前循环每次翻转后 HAL_Delay(300)，因此一个完整周期包含两次约 300 ms 延时。将单次延时改为 500 ms，预计完整周期约 1000 ms，可满足 ±50 ms 目标。构建成功和烧录 verified 仅证明固件已成功生成并写入，不代表硬件周期通过；UART 的 partial_tail 仅表示采集结束时最后一行缺少换行，不是固件故障。修改后仍需由 PL 重新测量 PA1 判定。

随后记录 source_diff 审批通过、应用文件、第二轮四工具重新执行及 PL 判 PASS。完整替换内容、diff 与源码 SHA 保存在公开 JSON 和差异文件中；本文展示工程理由，不提供 API Key 或认证头。现有提示模板、LLM 返回、审批记录和工具结果用于核对本节概要；未保存的 HTTP 报文不重建为所谓原始请求。

人工采纳这一修改的依据，是完整周期包含两次翻转间隔，首轮 PL 周期及高电平数据与源码 HAL_Delay(300)相互印证；人工批准该修改，并非因为 LLM 预计其能够通过，而是因为其推理与源码结构及首轮物理证据相互印证。审阅对象为完整 diff：除 300→500 外，还包含 assert_failed 参数注释“error line source number”→“line source number”的调整，均按实际批准记录保留。第二轮独立 PL 测得 1001.08682 ms 才形成目标通过证据，不能把模型预计的 1000 ms 写成实测。

3.3.6.3 未采纳内容与取舍理由

当前主 JSON 记录的一项 LLM 源码修改提案经人工审批后已采纳，未记录可核验的被人工否决的 LLM 源码修改提案。因此，本节不虚构所谓“模型建议被拒绝”的历史记录，仅列出能够由实现、测试和现有材料支持的工程取舍。

表 3-8 未采纳内容与取舍理由

未采用的路线 / 判断	人工取舍理由	当前实现或验证依据
让 LLM 自行批准修改并直接执行硬件命令	推理建议与执行权限分离，源码变更必须经人工审阅；范围、顺序及预算由确定性程序约束	controller.py、approval.py、orchestrator.py；主 JSON 的 reviews 及 applied_edit
仅凭 LLM 判断、UART 打印或烧录 verified 判定目标达标	三者分别反映建议、软件自报或写入状态，不能代替 PA1 真实周期	主 JSON 第一轮四工具 success 但物理 FAIL，第二轮 PL 才 PASS
将正常 UART 窗口末行半行一律当作传输截断	正常 deadline 不等于证据丢失；仍需严格处理 overflow、capture_limit 及真实解码错误	tools/pl_uart.py、UART 窗口专项回归和主 JSON 第一轮 partial_tail=true、truncated=false

上述区别也限定了证据的使用方式：离线负向测试可以说明约束在相应条件下如何生效，但不能据此声称真实硬件运行中曾发生过相同故障，也不能补写并不存在的“人工拒绝 LLM 提案”案例；历史故障的修复同样不能自动证明所有新平台均已通过验证。

3.3.6.4 人工凝练的协作心得

本次 LLM 协作中，人的关键工作不是接受模型给出的答案，而是将其工程建议转化为可检查、可验证的假设：先确认测量对象是完整周期，再核对两次延时与独立物理观测是否一致，审阅完整 diff，最后重新执行真实硬件测量。LLM 的合理解释只获得试验资格，实际物理结果才决定是否能够形成已验证结论。

记录未采用路线及其理由，可以保留设计边界；进一步将失败原因、判断依据、验证方法与失效条件整理为 Skill，则可以使一次具体的 LLM 协作经验被后续硬件任务复用。单次两轮闭环通过不能推出普遍成功率，也不能将未保存的对话或未发生的审批过程补写为原始记录。

3.3.7 Skill 提炼过程与验证

本项目按照可复用能力，**将经验组织为“一个总闭环、三类支撑能力”的层次结构。**其目的不是增加 Skill 文件数量，而是让陌生使用者能够判断何时使用、需要替换哪些平台假设、如何验证结果以及何时应停止使用该方法。

其中，hardware-debug-workflow 是总控 Skill，描述从可量化目标、真实工具执行、证据获取、确定性判定，到 AI 提出修改、人工审批及重新验证的完整

闭环。在此基础上，pl-observation 与 uart-observation 组成可信证据层，分别解决物理观测 freshness、一致性以及 UART 窗口和健康状态问题；deployment-and-flashing 构成可靠部署层，用于保证源码、新构建产物、远端文件与实际烧录对象之间的身份一致；failure-diagnosis 构成故障诊断层，用于区分工具故障、访问故障和 DUT 本身未达标，避免让 AI 通过修改源码掩盖基础设施问题。

这些 Skill 均按照“失败或观察 → 可复用规则 → 验证方法 → 迁移条件 → 失效边界”的方式提炼。例如，UART 窗口末尾正常半行曾被误判为证据截断，由此形成 partial_tail 与真正 truncated 分离的处理规则；PL 寄存器可能保留旧测量值，则进一步形成新边沿、valid 与一致快照联合判断的 freshness 规则。它们并非从一次成功演示反向编写，而是由真实开发过程中的失败和边界逐步凝练。

复用时，使用者不应直接复制本项目的管脚、寄存器地址或工具路径，而应保留方法本身并替换部署假设。例如迁移 pl-observation 时，需要重新确认目标时钟、寄存器映射、输入管脚与计数范围；迁移 deployment-and-flashing 时，需要替换目标 MCU、工具链、烧录器及对应配置。迁移完成后仍需在新的真实硬件上重新获得验证结果，本项目的通过记录不能替代新平台验收。

因此，本项目所提交的 Skill 更接近一组可迁移的硬件调试方法，而不是针对“智调盒”本身编写的操作说明。其共同原则是：模型可以提出下一步假设，但工具身份、物理证据和最终 PASS/FAIL 必须能够由独立、可检查的工程过程确认。[1]

表 3-9 Skill 提炼过程与验证。五项 Skill 并非彼此独立：hardware-debug-workflow 作为总闭环，调用可信证据、可靠部署与故障诊断三类支撑能力，共同保证一次 AI 硬件调试既能执行，也能证明其结果可信。

层级	主题 Skill	解决的问题	迁移时需替换	验证依据	失效边界
总闭环	hardware-debug-workflow	将“目标定义—工具执行—真实证据—确定性判定—AI 修改建议—人工审批—重新验证”组织为受控闭环，避免模型仅凭自身判断宣布硬件 PASS。	被测目标及容差、目标工程与允许修改文件、工具后端、物理观测方式、最大轮次与预算、人工审批边界。	主实验两轮完整 JSON；第一轮物理 FAIL、AI 提议修改、人工批准、第二轮重新执行并 PASS；workflow 相关回归测试。	缺少独立物理观测；模型获得越权执行或自审批能力；工具链、时间预算或最大轮次不足以完成可靠验证。
可信证据层	pl-observation	防止旧寄存器值或历史测量冒充本轮新结果，通过新边沿、有效位和一致快照确认物理证据 freshness，并完成计数到时间参数的确定性换算。	PL 时钟频率、寄存器映射、寄存器位宽、输入管脚、计数范围、被测参数及换算关系。	主实验两轮均记录 fresh_edges=2、freshness_verified=true；周期/高电平实测结果；PL reader 离线回归。	未确认当前 PL 镜像或时钟；计数超出可解释范围；无新边沿；启动阶段首次残段等已知边界未处理。
	uart-observation	区分正常采集窗口结束时的末行半行与真正的数据截断或通信错误，避免把辅助日志误判为致命故障，同时保留健康状态。	UART 基址、波特率、数据格式、FIFO/状态寄存器语义、采集窗口、字节上限及错误标志定义。	主实验第一轮真实出现 partial_tail=true、truncated=false 后仍继续 PL 测量；UART 专项与 workflow 回归。	不豁免 overflow、capture_limit、真正截断、解码/帧/校验错误；换波特率或接口后必须重新验证。

层级	主题 Skill	解决的问题	迁移时需替换	验证依据	失效边界
可靠部署层	deployment-and-flashing	保证“刚刚构建的产物”与“真正写入目标板的固件”身份一致，避免旧HEX、上传错误或远端文件不一致污染后续观测。	编译工具链、目标 MCU、工程入口、产物格式、烧录器、OpenOCD target/interface、远端路径与传输方式。	构建后新产物 SHA、SCP 后远端 SHA 一致性、OpenOCD program / verify / reset 记录，以及真实两轮烧录日志。	SSH host key、工具依赖或来源未确认；目标器件/烧录配置不匹配；产物 SHA 不一致；不能验证实际烧录对象身份。
故障诊断层	failure-diagnosis	在失败后先区分“工具/基础设施故障”与“DUT 物理目标未达标”，再决定停止、恢复或允许 AI 分析源码，避免错误地通过改代码掩盖访问故障。	平台错误码、设备节点、网络与工具健康检查、超时策略、恢复步骤、可重试/不可重试条件。	BusyBox、模型预算、取消、SSH/访问等实际开发问题的归纳；Controller 与负向回归；主实验中工具 success 与 DUT passed 分离的真实案例。	无法确认故障来源时不得直接归因于 DUT；不能让 AI 修改源码来掩盖烧录、网络、权限、设备访问或基础设施故障。

当前五项 Skill 不依赖 PYNQ 框架，因此本文不主张通用 PYNQ Skill 的额外加分；其中 pl-observation 已将寄存器映射、freshness 校验与结构化观测解耦为可迁移方法，后续可进一步适配 PYNQ Overlay 与 register_map 接口。

3.3.8 从零复现说明

本文的复现从准备已交付工程及明确外部依赖开始，依次验证软件、部署已知系统、准备 v5 并运行新任务；固件、FPGA 和 OpenOCD 的重新构建按各自入口执行。Linux 按已采用的预构建镜像 rootfs 定制路线操作，不要求补做完整 PetaLinux 源工程。基底镜像及合法外部 IP 不因本报告存在而自动获得再分发许可，具体版本 / SHA 和范围见附录 A、平台及依赖清单。

步骤一，在新目录取得固定版本；现有工作副本应先保留本机配置及未提交

工作。示例：

```
git clone https://github.com/Aurora-wwj/zhitiao_box.git
cd zhitiao_box
git checkout 9f2e1bb71a072e667a3ae55777ca097deface748
git status --short
cd src/agent
$env:PYTHONPATH = (Resolve-Path .\src).Path
py -3.13 -B -m unittest discover -s tests -v
.\start_demo.bat
```

步骤二，在仓库根运行 python3.13 -B sim/check_release.py。根 sim 的完整 32 项需 Python3.13、GCC / 主机 C 编译器、Icarus Verilog、Tcl、U-Boot 工具 mkimage / dumpimage 及 dtc；前端 Agent 专项另需 Node。缺少工具的测试会显示 skip，不能当作已实际执行。软件演示通过后再进入真实部署；当前结果以对应提交和实际输出为准，不沿用历史 Windows 截图计数。

步骤三，按 2.3.9 从合法配套介质准备已知 image.ub 及原 BOOT.BIN，核对输入 SHA；在 Linux / WSL 中执行离线定制，工作目录必须不存在：

```
python3.13 -B build/linux/customize_rootfs.py inspect \
  /path/to/image.ub
python3.13 -B build/linux/customize_rootfs.py repack \
  /path/to/image.ub --output build/linux/work-rootfs
```

这里的路径为待替换示例；输出目录由 Git 忽略，不把原件、账号数据库、现场私钥或新镜像上传 Git。检查 build_result.json、组件 SHA、唯一网络改动及 mkimage.log；在备用 SD 按 board/zynq/sd_deployment.md 安装候选 image.ub，保留原 BOOT 与回退文件，并记录串口、网络、SSH、USB 及 SWD 复验。新产物未经过自己的启动验收前，不用历史 fixed 报告替它证明。

步骤四，Windows 安装 Keil、Arm Compiler V5.06 update6 与 STM32G4xx DFP2.0.0，选原工程 src/stm32/gpio_pl_test、入口 mdk-arm/gpio_pl_test.uvprojx、Target GPIO_PL_Test。根 build/stm32/build.ps1 接收 UV4.exe 绝对路径并检查新产物。LED 变体另选 gpio_pl_test_led，使用独立 deployment.local.led.json 及自己的输出 HEX，不能把 build/stm32/firmware.hex 当成其产物。

步骤五，若独立重建 FPGA，使用 Vivado2017.4 及四项合法外部 IP，先运行 build/fpga/check_ip_repo.py 核对 VLNV 和 37 个文件，再运行 build/fpga/build.tcl。当前 HDF / v5 是已交付身份一致的产物；实际新重建、时序覆盖及上板结果仍需本机记录。重建后的位流若 SHA 不同，按新输入重新批准和验收，不覆盖历史黄金文件冒充原件。

步骤六，按接线图准备 Zynq、STM32 与 ST-Link。板端使用已验收 ARM32 OpenOCD 及完整 scripts；build/openocd/build.py 提供交叉构建入口，已有 PC --native 只证明配方和新生成 x86-64 程序可执行，不能把它装到 Zynq。确认/sd 挂载、静态 IP、可信 known_hosts 和公钥登录。版本用 uname、/proc/version、BusyBox、/etc/issue 核对，不能因缺/etc/os-release 就判系统失败。

步骤七，严格按当前初始化顺序：Linux 和网络就绪→三步 SSH 公钥恢复 / 检查→v5 文件 SHA→手动加载至/dev/xdevcfg 并确认 prog_done→启动 Agent 与 LED 。 PYTHONPATH 、 SSH 恢 复 和 cat 加 载 命 令 完 整 保 留 在 board/zynq/pl_preparation.md，本仓库使用 src/agent 相对路径，已有 E 盘独立部署例子不要求照抄。check_only 不自动加载，当前 BusyBox 无 timeout，不直接切换 load_verified；不重建 BOOT.BIN。

步 骤 八 ， 在 src/agent 复 制 deployment.example.json 为 私 人 deployment.local.json，填写本机路径并核对 UART / PL / LED 配置。主示例和 deployment-setup 向导已统一 v5＋LED，--no-led 或 deployment.no-led.example.json 只关闭状态灯，仍保留同一 v5、UART 和 PL；未配置 status_indicator 的旧配置继续不写灯。已有配置不被自动覆盖。先预览，再由可信终端批准实际操作：

```
py -3.13 -B -m zhidiao_agent deployment-check `
  --config deployment.local.json --preview-only
py -3.13 -B -m zhidiao_agent real-log `
  --config deployment.local.json --duration-s 5 --preview-only
py -3.13 -B -m zhidiao_agent real-test `
  --config deployment.local.json `
  --expected-period-ms 1000 --tolerance-ms 50 --preview-only
```

步骤九，启动 start_ui.bat，选择工程、core/src/main.c 和模型，保留 UART 启用，在本机输入 Key 并批准任务范围和每个新 diff。仓库原黄金 main 和 LED 变体均已是 500 ms 延时，基线直接 PASS 属于正常行为；若复现 FAIL→修改→PASS，应在独立副本准备已知未达标基线并重新配置审批根路径，不为展示去改黄金证据。

步骤十，下载新的 JSON 并核对同轮四工具 audit、UART 健康、freshness、物理容差和源码 / 产物 SHA。记录提交号与工作区状态，在 src/agent 运行 scripts/test_windows.ps1 保存最新 Windows 结果；不要重放 data 内的旧批准。当前模型调用属于真实联网和计费请求，按控制台的同意 / 审批执行，不将离线演示结果作为新真机通过。

3.3.9 HDF 与 Linux 定制的离线验证

本节引用固定提交内已归档的离线执行记录，不把文档制作重复统计为新实验。已对用户 HDF 原件及内部 10 个成员完成 SHA、CRC、器件 / 工具版本和寄存器检查，内含 v5 bit 及 BD 与当前黄金文件一致。该证据验证交接文件身份，不等同于重新运行 Vivado、生成 FSBL 或重建 BOOT。检验入口为 sim/linux_check.py 及 test_linux_release.py。

对收到的原 image.ub 在两个新目录实际重封装，使用 Python3.13.5、mkimage2025.01 和 dtc1.7.2，两个新 FIT 逐字节一致，大小 9537626 字节。标准 dumpimage 独立提取三个载荷与解析器结果相符；kernel、FDT 原字节保持，964 份原 CPIO 成员中仅 etc/network/interfaces 改变，其余记录及元数据保持。新镜像整体 SHA 与历史 fixed 不同，记录见附录 A；原因包括不复制历史私钥 / terminfo 变动以及独立压缩、封装元数据。不能把相同目标配置等同于相同镜像字节。

表 3-10 HDF 与 Linux 定制的离线验证

核对项	实际结果	验证边界
HDF / v5	10 成员、CRC、内含 bit 和 BD 一致	文件身份与设计参数；不是新硬件重建
kernel / FDT	新旧载荷逐字节相同	保持旧版本，没有重编译内核或设备树
CPIO	964 成员中仅网络配置改变	其他文件 / 设备节点 / 链接 / 权限保持
重封装重复性	两次输出整体字节与 SHA 相同	同一主机工具环境；跨版本整体 SHA 可能不同
输入与错误路径	坏 HDF、坏 FIT、未知基底、已有输出等拒绝	6 项新增 Linux / HDF 回归；未运行 ARM 镜像
启动与真实链路	队伍报告原系统冷启动链路已验	新生成镜像 board_tested=false，需自己的备用卡记录

上述 6 项包含在根 sim32 项中，不与 32 再次相加。完整命令及 SHA 见 data/linux_customization_verification.txt 和 linux_repack_check.json；同一 Git 树的干净导出副本已执行 32 项且无 skip。

第四部分　总结

4.1 可扩展之处

当前智调盒已围绕 GPIO 周期测量与 UART 日志建立真实硬件闭环。后续可进一步支持 PWM、ADC、SPI、I²C 等信号与协议，并接入更多 MCU、烧录工具与硬件 Backend，同时完善 PL 自动加载、部署初始化、接口标识与异常恢复机制，降低跨平台部署成本。在观测能力上，可由单一周期、高电平等参数测量扩展到更完整的波形采集、特征提取与异常识别，使 AI 获得更丰富的物理证据；在交互方面，可增加 LCD、OLED 或 HDMI 等显示方式，对波形、目标容差及 Before / Target / After 结果进行可视化。最终形成可迁移、可扩展的 AI 硬件调试方式。

4.2 心得体会

回顾整个设计过程，我们发现真正困难的，是**让一个系统有办法自己回答：某个修改真的起作用了吗？**

这也成为智调盒最核心的设计原则——模型建议、工具执行成功与物理目标通过，必须是三个不同的概念。AI 可以提出代码修改方案，烧录工具可以证明新的固件已写入目标板，但都不能代替最终判断。只有由 PL 独立测得 1001.08682 ms，并满足目标容差，这次修改才真正完成了从“合理建议”到“真实结果”的跨越。这也是本项目的构想与 FPGA 平台天然适配的最主要原因。

真实硬件也不断提醒我们，许多问题并不会出现在理想流程里。寄存器中存在数值，却可能只是上一轮留下的旧数据；模型调用的超时、烧录器的状态、网络连接和一次物理接线，都可能使完整闭环停在某一步。正是这些具体而琐碎的问题，使我们逐渐建立了 freshness、健康状态、超时预算、停止条件与异常恢复等机制。**相比一次顺利的 PASS，那些失败反而更清楚地告诉我们，一个能够接触真实世界的 AI 系统还需要什么。**

我们也重新理解了“自动化”和“人工参与”的关系。智调盒并没有试图把人从系统中彻底删除，Controller 可以按照确定规则组织工具执行，但新的源码 diff 仍需要人工审批。**人的作用不只是按下一个确认按钮，而是在关键边界上决定什么可以被执行、什么证据值得相信，以及一次看似合理的修改是否真正具有验证价值。**

与 AI 协作的过程同样改变了我们看待“大模型辅助开发”的方式。模型最有价值的地方并不是永远给出正确答案，而是能够依据返回的新证据提出下一步假设。首轮 FAIL 并不是系统的失败，它恰恰构成了闭环最重要的一半：**只有系统能发现自己没有达到目标，修改后的 PASS 才真正有意义。**

我们也把开发过程中遇到的失败、判断依据与验证方法整理成了可复用的 Skill。这样，一次具体的调试经验就不再只属于某一次实验，而有机会转化为下一次硬件任务仍可使用的方法。

未来，智调盒仍有许多可以继续完成的工作：更丰富的物理接口、更广泛的目标平台、更自动化的部署流程，以及更复杂的硬件测量与调参任务。但我们希望它所表达的核心思想已经足够清晰：**AI 不应只停留在生成代码的屏幕里，而应能够走向真实硬件，并接受真实物理世界的验证。**

希望“智调盒——AI 上板摆渡站”这个项目的尝试，哪怕只是很小的一步，也能够触碰到嵌入式开发新范式的一角。

也感谢每一位愿意看到这里、愿意了解这个想法的人。希望这个尚不完美、却真实运行在硬件上的作品，能够让您看到我们所期待的那种可能。

第五部分 参考文献

[1] 全国大学生嵌入式芯片与系统设计竞赛组委会. 全国大学生嵌入式芯片与系统设计竞赛 2026 FPGA 赛道选题指南（AMD）[R]. 2026.

[2] AMD. Zynq-7000 SoC Technical Reference Manual: UG585[R/OL]. AMD 官方文档中心, https://docs.amd.com/（按文档编号检索）.

[3] Xilinx. Vivado Design Suite User Guide: Synthesis, UG901, v2017.4[R]. 2017.

[4] Xilinx. Vivado Design Suite User Guide: Implementation, UG904, v2017.4[R]. 2017.

[5] Xilinx. Vivado Design Suite User Guide: Using Constraints, UG903, v2017.4[R]. 2017.

[6] AMD/Xilinx. AXI GPIO v2.0 Product Guide: PG144[R/OL]. AMD 官方文档中心, https://docs.amd.com/.

[7] AMD/Xilinx. AXI UART Lite v2.0 Product Guide: PG142[R/OL]. AMD 官方文档中心, https://docs.amd.com/.

[8] STMicroelectronics. STM32G431x6/x8/xB Datasheet: DS12589[R/OL]. ST 官方技术文档中心, https://www.st.com/.

[9] STMicroelectronics. STM32G4 Series Advanced Arm-based 32-bit MCUs Reference Manual: RM0440[R/OL]. ST 官方技术文档中心, https://www.st.com/.

[10] STMicroelectronics. STM32CubeG4 MCU Package[CP/OL]. https://github.com/STMicroelectronics/STM32CubeG4.

[11] OpenOCD Developers. Open On-Chip Debugger User's Guide, 0.12.0[EB/OL]. https://openocd.org/doc/html/index.html.

[12] Arm. CMSIS Version 5 Documentation[EB/OL]. https://arm-software.github.io/CMSIS_5/General/html/index.html.

[13] ALINX. AX7Z020 开发板及配套参考工程资料[CP/OL]. https://www.alinx.com/（按板卡型号获取配套版本）.

[14] 本作品工程与验证材料：代码与证据基准提交 9f2e1bb71a072e667a3ae55777ca097deface748，run_5d8e004954664f6c8f8ff0f85d31959b 及公开代码、清单、报告与原图[CP/OL]. 2026. https://github.com/Aurora-wwj/zhitiao_box/tree/9f2e1bb71a072e667a3ae55777ca097deface748.

[15] U-Boot Project. Flattened Image Tree（FIT）文档[EB/OL]. https://docs.u-boot.org/en/latest/usage/fit/index.html. 本文封装工具实际版本见 data/linux_repack_check.json，不据此升级目标板 U-Boot。

附录 A　素材与证据路径清单

下列路径均相对固定提交 9f2e1bb71a072e667a3ae55777ca097deface748 的仓库根目录。本文的原始图、数字和代码关联不包含学校、队伍、成员、认证或私钥内容；原运行报告的本机路径按公开副本规则脱敏，历史审批只作审计。完整正文到代码对照见附录 B，电子核验清单逐项保存路径及该提交的文件 SHA。

表 A-1 附录 A　素材与证据路径清单

材料路径	支撑内容
data/final_run_sanitized.json	主实验两轮四工具、模型提案、批准、UART 及 PL
data/report_sources.json	同 run 不同导出与四张原图 SHA、场景关联
data/measurements.csv；data/final_source_change.diff	数值表和实际批准差异的完整 LF 副本
data/screenshots/	四张原始网页 PNG；不是实物 / 接线照片
src/stm32/gpio_pl_test/core/src/main.c	原成功工程最终 main，映射历史 Core/Src 路径
build/stm32/firmware.hex；data/stm32_sha256.json	第二轮固件身份与当前原工程清单
src/stm32/gpio_pl_test_led/	独立八灯同步演示工程；不借用主实验 JSON
data/stm32_led_manifest.json；data/stm32_led_verification.txt	变体来源、双 SHA 及主机验证边界
data/repository_path_map.json	123 份路径迁移、当前 SHA 及 24 份不变证据
src/fpga/zhidiao_measure/	RTL、BD、XDC、XPR、Tcl 与当前 HDF
build/fpga/reports/；data/fpga_metrics.json	baseline / final 原实现报告与解析指标
build/fpga/zhidiao_observe_v5_led.bit	黄金 v5 位流
build/fpga/build.tcl；data/fpga_external_ips.json	重建入口、VLNV 与 37 文件外部依赖

材料路径	支撑内容
data/linux_platform.json	系统版本来源、历史镜像 SHA、HDF 成员和范围
build/linux/customize_rootfs.py ；build/linux/readme.md	受限原基底的网络定制与 FIT 重封装方法
board/zynq/rootfs_network.patch	真实原基底生成的网络差异
board/zynq/sd_deployment.md ；board/zynq/pl_preparation.md	SD 部署、SSH 和 v5 手动初始化、冷启动复验
data/linux_repack_check.json	两次完整重封装、组件与 CPIO 独立比较结果
data/linux_customization_verification.txt	实际命令、工具版本、32 项与未执行范围
data/zynq_linux_manifest.json	原两份 FIT 组成、OpenOCD 来源与排除说明
build/openocd/ ；src/third_party/openocd/	ARM 工具构建配方、源码 / 许可证；PC 验证边界
skill/	五类主题 Skill、适用 / 来源 / 验证 / 失效条件
data/test_summary.md ；data/path_migration_verification.txt	测试批次、465 项与新路径源文件验证记录

本表按仓库根目录列出实际路径，电子对照清单逐项核对。未放入 Git 的 BOOT / 镜像 / 现场私钥不因出现在表中就视为已公开发布；已收原件身份由对应清单提供，获得与使用遵循供应商配套资料及许可边界。

表 A-2 附录 A 素材与证据路径清单

黄金对象 / 已核验本地产物	SHA-256
原工程最终 main.c	79f6deb58231d3bc2cef887154c136a74e0da9e839a26ea41fd9468e205633cd
第二轮 HEX	b6aca82331978cca76e967fd7d08f3d33994672215ff3e6a50f0173662b9f70f
v5 bitstream	fb4d05e6a075bd405ffa14c1d1a6631fa0a42e4833999c0a0f24b92b027118d6
当前 HDF	ff24021b5052bf91d5101e804fdd2917c457fc8a45db16caed89d349c3fd3198
LED 同步版 main.c	053697e2f914f435bae950c1529dd6b68af01532c12b94318ae622115a7debfd

黄金对象 / 已核验本地产物	SHA-256
原 image.ub	224b9b884352a931244541017024fdbff69297feeaadab2b1e5eb0d030bb0344
历史 image_fixed.ub	71fb28754b857b808340f87e9b54de2197d54141fd4c90eb1fd94360ac78635a
本次离线新镜像	28ca54d63467a2ec73d23e0cd31bf46fe8107a0966a2aec759e8fe92c6daaa45

来源包标识和 known_good 的 SHA 按已有原件登记。原系统与新产物证据分开，时序 / 启动边界、免密持久化和 PL 自动加载不推定通过；不要求重复上传 SD 或补做完整 PetaLinux 工程。

附录 B　文档、代码与验证证据对照

每行将正文主题对应到本固定提交中可读取的实现 / 清单及验证入口。实现源文件证明功能如何编写，测试说明离线覆盖，真实 JSON 证明当次运行；三类证据不能互相替代。文件路径和 SHA 由生成时的 Git 对象核对。

表 B-1 附录 B　文档、代码与验证证据对照

正文范围	实现 / 材料路径	核对入口或证据
1.1～1.7；2.1	README.md ；report/project_overview.md	data/final_run_sanitized.json ；report/test_evidence.md
2.2.1～2.2.3	board/wiring.md ；src/fpga/zhidiao_measure/system_bd.tcl ；src/fpga/zhidiao_measure/linux_ov5640.srcs/constrs_1/new/zhidiao_measure.xdc	data/fpga_manifest.json ；sim/fpga_check.py
2.2.4 及四级小节	src/fpga/zhidiao_measure/linux_ov5640.srcs/sources_1/new/zhidiao_measure_core.v ；src/agent/src/zhidiao_agent/tools/pl_observer.py	sim/fpga/measure_core_tb.v ；src/agent/tests/test_pl_observer.py
2.2.5 及四级小节	src/agent/src/zhidiao_agent/tools/pl_uart.py ；src/agent/src/zhidiao_agent/deployment.py	src/agent/tests/test_uart_window_boundary.py；data/final_run_sanitized.json
2.2.6；2.3.8	src/agent/src/zhidiao_agent/status_indicator.py ；src/agent/src/zhidiao_agent/events.py	src/agent/tests/test_status_indicator.py ；src/fpga/zhidiao_measure/linux_ov5640.srcs/constrs_1/new/zhidiao_measure.xdc
2.2.7～2.2.8	data/linux_platform.json ；src/fpga/zhidiao_measure/system_wrapper.hdf	sim/linux_check.py ；sim/test_linux_release.py
2.2.9	src/stm32/gpio_pl_test_led/core/src/main.c ；data/stm32_led_manifest.json	sim/stm32_led_check.py ；sim/test_stm32_led_release.py

正文范围	实现 / 材料路径	核对入口或证据
2.3.1～2.3.3	src/agent/src/zhidiao_agent/agent.py；src/agent/src/zhidiao_agent/orchestrator.py；src/agent/src/zhidiao_agent/controller.py	src/agent/tests/test_v05_workflow.py；data/final_run_sanitized.json
2.3.4 及四级小节	src/agent/src/zhidiao_agent/tools/interfaces.py；src/agent/src/zhidiao_agent/tools/real_backend.py	src/agent/tests/test_real_backend.py；src/agent/tests/test_real_workflow.py
2.3.5	src/agent/src/zhidiao_agent/tools/pl_prepare.py；src/agent/src/zhidiao_agent/tools/pl_observer.py；src/agent/src/zhidiao_agent/tools/pl_uart.py	board/zynq/pl_preparation.md；src/agent/tests/test_preparation_workflow.py
2.3.6	src/agent/src/zhidiao_agent/approval.py；src/agent/src/zhidiao_agent/session.py；src/agent/src/zhidiao_agent/editing.py	src/agent/tests/test_approval.py；src/agent/tests/test_editing.py；data/final_source_change.diff
2.3.7	src/agent/src/zhidiao_agent/desktop.py；src/agent/src/zhidiao_agent/web_ui.py；src/agent/src/zhidiao_agent/ui_page.py	src/agent/tests/test_ui_report_identity.py；src/agent/tests/test_desktop_sources.py
2.3.9 及四级小节	build/linux/customize_rootfs.py；board/zynq/sd_deployment.md	data/linux_platform.json；data/linux_repack_check.json
3.1～3.2	report/material_index.md；report/release_status.md	data/report_sources.json；data/repository_path_map.json
3.3.1～3.3.2	data/final_run_sanitized.json；data/measurements.csv	sim/report_check.py；sim/check_release.py
3.3.3	data/test_summary.md；sim/test_linux_release.py	data/path_migration_verification.txt；data/linux_customization_verification.txt

正文范围	实现 / 材料路径	核对入口或证据
3.3.4～3.3.5	data/fpga_metrics.json ；build/fpga/reports/final/system_wrapper_timing_summary_routed.rpt	sim/fpga_check.py ；data/final_run_sanitized.json
3.3.6	src/agent/src/zhidiao_agent/agent.py ；data/final_source_change.diff	data/final_run_sanitized.json ；report/llm_collaboration.md
3.3.7	skill/README.md ；skill/hardware-debug-workflow.md ；skill/deployment-and-flashing.md	skill/pl-observation.md ； skill/uart-observation.md ； skill/failure-diagnosis.md
3.3.8	report/reproduction.md ；board/zynq/pl_preparation.md ； build/linux/readme.md	src/agent/src/zhidiao_agent/setup_cli.py；src/agent/deployment.example.json；src/agent/deployment.no-led.example.json
3.3.9	build/linux/customize_rootfs.py ；src/fpga/zhidiao_measure/system_wrapper.hdf	sim/test_linux_release.py ；data/linux_customization_verification.txt
4.1～4.2	report/release_status.md ；report/optimization_notes.md	report/fpga_evidence.md ；data/linux_platform.json